HyperionDev

| | |
|---|---|
| **Curriculum title** | Python Programmer |
| **Curriculum code** | 900221-000-00-00 |
| **Module code** | 900221-000-00-KM-03 |
| **NQF level** | 4 |
| **Credit(s)** | 4 |
| **Quality assurance functionary** | QCTO - Quality Council for Trades and Occupations |
| **Originator** | MICT SETA |
| **Qualification type** | Skills Programme |

# Principles of Programming with Python

## Learner Guide

# Table of contents


**Table of contents** **2**
**Introduction** **4**
**Purpose of the Knowledge Module** **4**
**Getting Started** **5**
**Knowledge Topic 1** **6**
1.1 Concept, definition and functions (KT0101) (IAC0101) 7
1.2 Syntax (KT0102) (IAC0101) 8
1.3 Flow charts (KT0103) (IAC0101) 11
1.4 Conditional statements: (KT0104) (IAC0101) 15
1.4.1 IF statements in Python 17
1.4.2 IF...else statements in Python 19
1.4.3 ELIF Statements in Python 23
1.5 For and while loops (KT0105) (IAC0101) 27
1.6 Break, continue and pass statements (KT0106) (IAC0101) 28
Formative Assessment Activity 1 30
**Knowledge Topic 2** **31**
2.1 Concept, definition and functions (KT0201) (IAC0201) 32
2.2 Syntax (KT0202) (IAC0201) 36
2.3 Types and uses (KT0203) (IAC0201) 39
2.3.1 Arithmetic Operators 46
2.3.2 Relational/comparison Operators 49
2.3.3 Assignment Operators 54
2.3.4 Logical/ Bitwise Operators 60
2.3.5 Membership Operators 66
2.3.6 Identity Operators 70
2.3.7 Operator precedence 76
Formative Assessment Activity 2 80
**Knowledge Topic 3** **80**
3.1 Concept, definition and functions (KT0301) (IAC0301) 82
3.2 Syntax (KT0302) (IAC0301) 84
3.3 Components (KT0303) (IAC0301) 88
3.4 Types of functions (KT0304) (IAC0301) 95
3.5 Built-in modules (KT0305) (IAC0301) 100
3.6 Main function and method: call, indentation, arguments and return values (KT0306) (IAC0301) 108
Formative Assessment Activity 3 114
**Knowledge Topic 4** **115**
4.1 Concept, definition and functions (KT0401) (IAC0401) 116
4.2 Syntax (KT0402) (IAC0401) 119





Copyright © 2024 HyperionDev. All rights reserved.

4.3 Dates, times and time intervals (KT0403) (IAC0401) 122
4.4 Datetime module (KT0404) (IAC0401) 129
4.5 Classes (KT0405) (IAC0401) 134
4.6 Timedelta objects (KT0406) (IAC0401) 141
Formative Assessment Activity 4 147
**Knowledge Topic 5** **148**
5.1 Concept, definition and functions (KT0501) (IAC0501) 149
5.2 Syntax (KT0502) (IAC0501) 154
5.3 Import (KT0503) (IAC0501) 159
5.4 Input/output (I/O) (KT0504) (IAC0501) 166
Formative Assessment Activity 5 173
**References** **174**



Copyright © 2024 HyperionDev. All rights reserved.

# Introduction

Welcome to this skills programme!

This guide will help you understand the content we will cover. You will also complete several class activities as part of the formative assessment process. These activities give you a safe space to practise and explore new skills.

Make the most of this opportunity to gather information for your self-study and practical learning. In some cases, you may need to research and complete tasks in your own time.

Take notes as you go, and share your insights with your classmates. Sharing knowledge helps you and others deepen your understanding and apply what you've learned.

# Purpose of the Knowledge Module

The main focus of the learning in this knowledge module is to build an understanding of the principles of programming with Python programming language.

This learner guide will enable you to gain an understanding of the following topics. (The relative weightings within the module are also shown in the following table.)

| Code | Topic | Weight |
|---|---|---|
| KM-03-KT01 | Making decisions in Python | 30% |
| KM-03-KT02 | Operators | 30% |
| KM-03-KT03 | Functions in Python | 15% |
| KM-03-KT04 | Date, Time and Calendar | 15% |
| KM-03-KT05 | Import, input and output | 10% |



Copyright © 2024 HyperionDev. All rights reserved.

# Getting Started

To begin this module, please click on the link to the **Learner Workbook** or download the PDF to easily access the content and obtain your workbook. This will prompt you to make a copy of the document, where you will complete all formative and summative assessments throughout the module. Ensure you save your copy in a secure location, as you will need to access it frequently.

Once you have made a copy, you can start working through the module materials. **Complete the assessments after reading and mastering the content covered in this module**. For each knowledge topic within the workbook, you may request a review of your work for assessment. Be sure to upload this document to your GitHub folder so your facilitator can review and mark it.



## Take note

This module has a total of 190 marks available. To meet the passing requirements, you will need to achieve a minimum of 143 marks, which represents 75% of the total marks. Achieving this threshold will ensure that you have met the necessary standards for this module.



Copyright © 2024 HyperionDev. All rights reserved.

# Knowledge Topic 1

| | |
|---|---|
| **Topic Code** | KM-03-KT01: |
| **Topic** | Making decisions in Python |
| **Weight** | 30% |

This knowledge topic will cover the following topic elements:

1.1 Concept, definition and functions (KT0101)

1.2 Syntax (KT0102)

1.3 Flow charts (KT0103)

1.4 Conditional statements: (KT0104)

   a. IF statements in Python

   b. IF...else statements in Python

   c. ELIF Statements in Python

1.5 For and while loops (KT0105)

   a. For loop in Python

   b. While Loop in Python

1.6 Break, continue and pass statements (KT0106)

   a. Using Break in For Loop

   b. Using Break Statement in While Loop

After working through this knowledge topic, your competence will be assessed according to the following criterion/criteria:

1. IAC0101 Definitions, functions and features of Python loops are understood and explained



Copyright © 2024 HyperionDev. All rights reserved.

## 1.1 Concept, definition and functions (KT0101) (IAC0101)

Making decisions in programming allows your code to execute certain actions only when specific conditions are met. This control flow mechanism enables programs to respond dynamically to different inputs and situations, making them more flexible and interactive.

### Definition

In Python, decision-making is handled using **conditional statements**. The primary conditional statements are:

- **if**: Checks a condition and executes a block of code if the condition is True.
- **elif** (short for "else if"): Checks another condition if the previous if condition was False.
- **else**: Executes a block of code if all preceding conditions were False.

### Function

These statements work together to control the flow of a program:

- **if Statement**: Initiates a decision branch.

```
if condition:
    # Code to execute if condition is True
```

- **elif Statement**: Adds additional conditions to check if previous ones failed.

```
if condition1:
    # Code if condition1 is True
elif condition2:
    # Code if condition2 is True
```

- **else Statement**: Catches all cases where previous conditions were False.


Copyright © 2024 HyperionDev. All rights reserved.

```
if condition:
    # Code if condition is True
else:
    # Code if condition is False
```

**Example:**

Let's see an example that decides whether a number is positive, negative, or zero.

```
number = 10
if number > 0:
    print("Positive number")
elif number == 0:
    print("Zero")
else:
    print("Negative number")
```

**Explanation:**

- The program checks if the number is greater than 0. If True, it prints "Positive number."
- If the first condition is False, it checks if the number equals 0. If True, it prints "Zero."
- If neither condition is True, it executes the else block and prints "Negative number."

By using these conditional statements, you can make your Python programs make decisions and perform different actions based on varying conditions.

## 1.2 Syntax (KT0102) (IAC0101)

Python uses conditional statements—if, elif, and else—to execute code based on certain conditions. Here's how you can use them:

**if Statement**


Copyright © 2024 HyperionDev. All rights reserved.

The if statement evaluates a condition and executes the indented code block if the condition is True.

```
if condition:
    # Code to execute if condition is True
```

**Example:**

```
age = 18
if age >= 18:
    print("You are eligible to vote.")
```

### elif Statement

The elif (short for "else if") statement checks another condition if the previous if (or elif) condition was False.

```
if condition1:
    # Code if condition1 is True
elif condition2:
    # Code if condition2 is True
```

**Example:**

```
score = 85

if score >= 90:
    print("Grade: A")
elif score >= 80:
    print("Grade: B")
```

### else Statement

The else statement executes a block of code if all preceding if and elif conditions are False.


Copyright © 2024 HyperionDev. All rights reserved.

```
if condition:
    # Code if condition is True
else:
    # Code if condition is False
```

**Example:**

```
temperature = 30

if temperature > 35:
    print("It's very hot.")
else:
    print("It's not too hot.")
```

### Combined Usage

You can combine if, elif, and else to handle multiple conditions.

```
if condition1:
    # Code if condition1 is True
elif condition2:
    # Code if condition2 is True
else:
    # Code if neither condition1 nor condition2 is True
```

**Example:**

```
number = 0

if number > 0:
    print("Positive number")
elif number == 0:
    print("Zero")
else:
    print("Negative number")
```

### Indentation Matters


Copyright © 2024 HyperionDev. All rights reserved.

Python relies on indentation to define code blocks. Each code block under if, elif, or else must be indented by the same amount (typically 4 spaces).

**Key Points:**

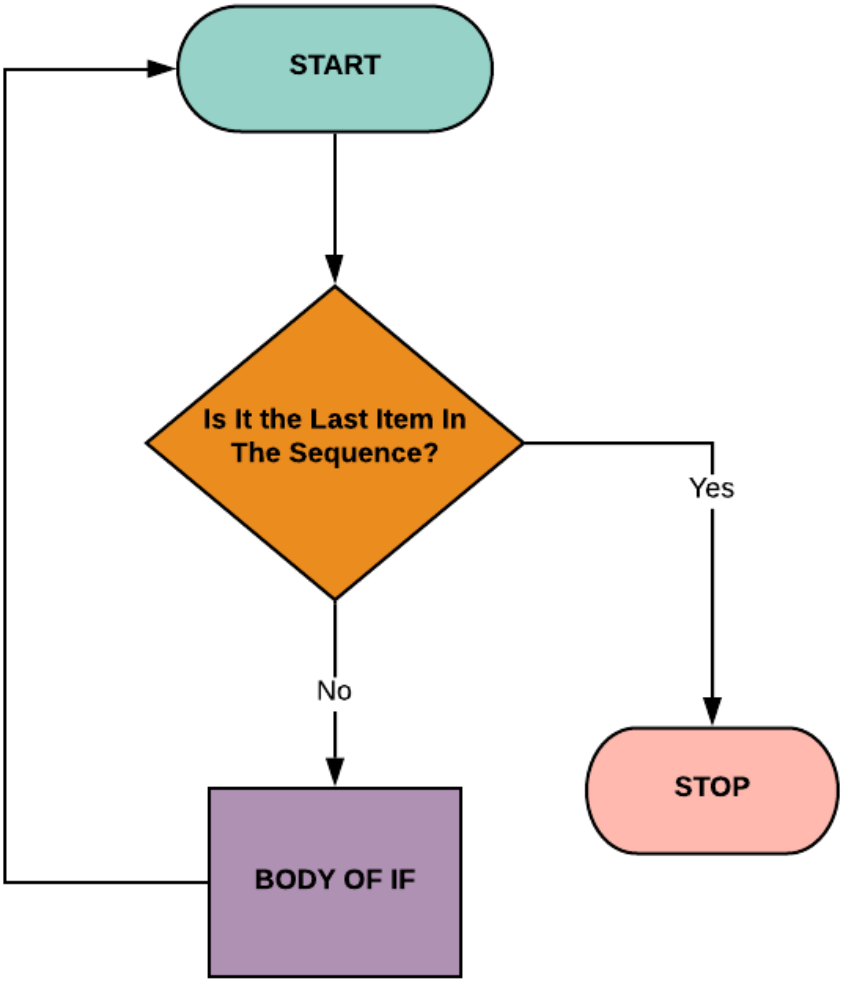
- **Conditions**: Expressions that evaluate to True or False.
- **Colons**: Use a colon (:) at the end of the if, elif, and else lines.
- **Indentation**: Indent the code blocks under each condition consistently.
- **Multiple Conditions**: Use elif for additional conditions and else as a fallback.

By using this syntax, you can direct your Python programs to make decisions and execute specific code based on varying conditions.

## 1.3 Flow charts (KT0103) (IAC0101)

A flowchart is a visual diagram that represents the sequence of steps and decisions needed to perform a process. In programming, flowcharts help illustrate the flow of control in code, making it easier to understand complex logic structures like conditional statements.


Copyright © 2024 HyperionDev. All rights reserved.




### Flowchart Symbols

- **Oval**: Indicates the start or end of a process.
- **Rectangle**: Represents a process or an action to be carried out.
- **Diamond**: Denotes a decision point that can lead to different paths based on a condition (True or False).
- **Arrows**: Show the direction of the flow from one step to the next.

### Using Flowcharts for Python's Conditional Statements

When writing decision-making code in Python using if, elif, and else statements, flowcharts can visually map out the logic flow. This helps in planning and debugging your code.


Copyright © 2024 HyperionDev. All rights reserved.

**Example Python Code:**

```
if condition1:
    # Execute block A
elif condition2:
    # Execute block B
else:
    # Execute block C
```

**Flowchart Representation:**

1. **Start**: Begin the process.
2. **Decision Point (Condition1)**:
    - **If True**: Execute **Block A**, then **End**.
    - **If False**: Proceed to next decision.
3. **Decision Point (Condition2)**:
    - **If True**: Execute **Block B**, then **End**.
    - **If False**: Execute **Block C**, then **End**.

**Visual Outline (Text-Based):**

- **Start**
    - Evaluate **Condition1** (Decision)
        - **Yes**: Perform **Block A**
        - **No**: Evaluate **Condition2** (Decision)
            - **Yes**: Perform **Block B**
            - **No**: Perform **Block C**
- **End**


Copyright © 2024 HyperionDev. All rights reserved.

### Benefits of Flowcharts in Python Decision Making

- **Clarifies Logic Flow**: Visual representation makes it easier to understand how conditions and actions are connected.
- **Simplifies Complex Decisions**: Breaks down complex conditional logic into manageable visual steps.
- **Enhances Communication**: Useful for explaining code logic to others, regardless of their programming expertise.
- **Aids in Debugging**: Identifies logical errors by tracing the flow of conditions and actions.

### Creating a Flowchart for a Python Program

1. **Identify All Conditions**: Determine all the if, elif, and else conditions in your code.
2. **Map Decision Points**: Use diamonds to represent each condition that needs evaluation.
3. **Draw Actions**: Use rectangles to denote the code blocks that execute for each condition.
4. **Connect with Arrows**: Indicate the flow from one step to the next based on the outcome of each condition.
5. **Start and End Points**: Clearly mark where the process begins and ends with ovals.

### Example Scenario: Checking a Number

Let's visualize a simple program that checks if a number is positive, negative, or zero.


Copyright © 2024 HyperionDev. All rights reserved.

**Python Code:**

```
number = int(input("Enter a number: "))

if number > 0:
    print("Positive number")
elif number == 0:
    print("Zero")
else:
    print("Negative number")
```

**Flowchart Steps:**

1. **Start**
2. **Input**: Get the number from the user.
3. **Decision (number > 0?)**
    - **Yes**: Print "Positive number" → **End**
    - **No**: Proceed to the next decision.
4. **Decision (number == 0?)**
    - **Yes**: Print "Zero" → **End**
    - **No**: Print "Negative number" → **End**

By using flowcharts to represent decision-making structures in Python, you gain a clearer understanding of how your program processes data and makes choices based on conditions. This visual tool is invaluable for both beginners learning programming logic and experienced developers designing complex systems.

## 1.4 Conditional statements: (KT0104) (IAC0101)

Conditional statements allow a Python program to execute specific code blocks based on whether certain conditions are met. They are essential for controlling program flow and implementing decision-making logic.


Copyright © 2024 HyperionDev. All rights reserved.

**Primary Conditional Statements**

1. **if Statement**
    - **Purpose:** Executes a block of code if a specified condition is True.
2. **elif Statement**
    - **Purpose:** Checks another condition if the previous if or elif condition was False.
3. **else Statement**
    - **Purpose:** Executes a block of code if all preceding if and elif conditions are False.

**Combining Conditional Statements**

- Multiple conditions can be handled using if, elif, and else together.

**Logical Operators in Conditions**

- **and**: True if both conditions are True.
- **or**: True if at least one condition is True.
- **not**: Inverts the Boolean value of the condition.

**Nested Conditional Statements**

- Conditional statements can be nested for more granular control:

**Conditional Expressions (Ternary Operator)**

- Python offers a one-line conditional expression for simple conditions:

**Important Notes**

- **Indentation**: Use consistent indentation (usually 4 spaces) to define code blocks.
- **Colons (:)**: Each if, elif, and else statement ends with a colon.
- **Conditions**: Must evaluate to Boolean values (True or False).

**Common Use Cases**


Copyright © 2024 HyperionDev. All rights reserved.

- **Decision Making**: Execute different code paths based on conditions.
- **Validation**: Check if data meets certain criteria before processing.
- **Control Flow**: Dynamically direct the flow of the program.

### 1.4.1 IF statements in Python

The if statement is a fundamental control structure in Python that allows programs to make decisions based on conditions. It enables your code to execute certain blocks only when specific conditions are met, controlling the flow of the program.

**Purpose**

- **Decision Making**: Execute code only if a particular condition evaluates to True.
- **Control Flow**: Direct the execution path of the program based on dynamic conditions.

**Syntax**

```
if condition:
    # Code to execute if the condition is True
```

- **if**: Keyword that introduces the conditional statement.
- **condition**: An expression that evaluates to True or False.
- **Colon (:)**: Indicates the start of the indented code block.
- **Indented Block**: Contains the code that executes when the condition is True. Python uses indentation (usually 4 spaces) to define this block.

**Example**

```
age = 18

if age >= 18:
    print("You are eligible to vote.")
```

**Explanation:**


Copyright © 2024 HyperionDev. All rights reserved.

- The condition age >= 18 is evaluated.
- If the condition is True, it prints "You are eligible to vote."
- If the condition is False, the indented code block is skipped.

### Multiple Conditions with if Statements

You can use multiple if statements to evaluate several conditions independently.

```
number = 7

if number > 0:
    print("Positive number")
if number % 2 == 1:
    print("Odd number")
```

**Output:**

```
Positive number
Odd number
```

### Logical Operators in Conditions

Combine conditions using logical operators:

- **and**: True if both conditions are True.
- **or**: True if at least one condition is True.
- **not**: Inverts the Boolean value of the condition.

**Example:**

```
temperature = 22
if temperature > 20 and temperature < 30:
    print("The weather is pleasant.")
```

### Nested if Statements


Copyright © 2024 HyperionDev. All rights reserved.

You can nest if statements inside other if statements for more granular control.

```
score = 95
if score >= 90:
    print("Excellent!")
    if score == 100:
        print("Perfect score!")
```

**Important Points**

- **Conditions Must Be Boolean**: The condition after if must evaluate to True or False.
- **Indentation Matters**: Indentation defines the scope of the code block under the if statement.
- **Colon (:) Usage**: Always place a colon at the end of the if statement line.

**Common Use Cases**

- **Input Validation**: Check if user input meets certain criteria before processing.
- **Feature Toggles**: Enable or disable features based on configuration or conditions.
- **Error Checking**: Validate data and handle exceptions appropriately.

### 1.4.2 IF...else statements in Python

The if...else statement is a fundamental control structure in Python that allows programs to make decisions and execute specific blocks of code based on whether a condition is True or False. It helps control the flow of a program by directing it to different execution paths.

**Purpose**


Copyright © 2024 HyperionDev. All rights reserved.

- **Decision Making**: Execute one block of code if a condition is True, and another block if the condition is False.
- **Control Flow**: Direct the program's execution based on dynamic conditions.

**Syntax**

```
if condition:
    # Code to execute if the condition is True
else:
    # Code to execute if the condition is False
```

- **if**: Introduces the conditional statement.
- **condition**: An expression that evaluates to True or False.
- **else**: Specifies the block to execute if the if condition is False.
- **Colon (:)**: Indicates the start of a new code block.
- **Indented Blocks**: The code under if and else must be indented consistently (usually 4 spaces).

**Example**

```
age = 16

if age >= 18:
    print("You are eligible to vote.")
else:
    print("You are not eligible to vote yet.")
```

**Explanation:**

- The condition age >= 18 is evaluated.
- **If the condition is True**, the program executes the code under the if block.
- **If the condition is False**, the program executes the code under the else block.

**Output:**


Copyright © 2024 HyperionDev. All rights reserved.

```
You are not eligible to vote yet.
```

### Key Points

- **else is Optional**: The else block runs only when the if condition is False.
- **Single else per if**: There can be only one else associated with an if statement.
- **No Condition for else**: The else block does not have a condition; it captures all cases not covered by the if condition.

### Using Logical Operators

Combine conditions using logical operators to form complex conditions.

- **and**: True if both conditions are True.
- **or**: True if at least one condition is True.
- **not**: Inverts the Boolean value of the condition.

**Example:**

```
password = "abc123"

if len(password) >= 8:
    print("Password length is sufficient.")
else:
    print("Password is too short.")
```


Copyright © 2024 HyperionDev. All rights reserved.

### Nested if...else Statements

You can nest if...else statements inside each other for more detailed decision-making.

```
number = 0
if number >= 0:
    if number == 0:
        print("Zero")
    else:
        print("Positive number")
else:
    print("Negative number")
```

**Explanation:**

- **First Condition**: Checks if the number is greater than or equal to 0.
    - **If True**:
        - Check if the number equals 0.
            - **If True**: Prints "Zero".
            - **If False**: Prints "Positive number".
    - **If False**: Prints "Negative number".

### Common Use Cases

- **Validation**: Check user input and provide appropriate feedback.
- **Decision Trees**: Execute different code paths based on multiple conditions.
- **Default Actions**: Provide a default action when the if condition is not met.

### Important Notes

- **Indentation Matters**: Consistent indentation defines the scope of the if and else blocks.
- **Colons (:)**: Both if and else statements end with a colon.
- **Boolean Conditions**: The condition must result in a Boolean value (True or False).


Copyright © 2024 HyperionDev. All rights reserved.

### 1.4.3 ELIF Statements in Python

The elif statement in Python, short for "else if," is a conditional control structure that allows you to check multiple conditions sequentially. It is used in decision-making to execute code based on which condition is True among several possibilities.

**Purpose**

- **Multiple Conditions**: Evaluate additional conditions if previous if or elif conditions are False.
- **Control Flow**: Direct the program to execute different code blocks based on multiple dynamic conditions.

**Syntax**

```
if condition1:
    # Code to execute if condition1 is True
elif condition2:
    # Code to execute if condition2 is True
elif condition3:
    # Code to execute if condition3 is True
else:
    # Code to execute if all above conditions are False (optional)
```

- **if**: Starts the conditional checking.
- **elif**: Follows an if or another elif, providing an additional condition to check.
- **else**: (Optional) Executes if none of the preceding conditions are True.
- **Conditions**: Expressions that evaluate to True or False.
- **Colons (:)**: Indicate the start of a new code block.
- **Indented Blocks**: The code under each if, elif, or else must be indented consistently.


Copyright © 2024 HyperionDev. All rights reserved.

**How It Works**

- The program evaluates the if condition first.
    - If True, it executes the code block under if and skips the rest of the elif and else statements.
    - If False, it moves to the next elif condition.
- Each elif condition is evaluated in order.
    - The first elif condition that is True will have its code block executed.
    - Subsequent elif conditions and the else block are skipped once a True condition is found.
- If none of the if or elif conditions are True, the else block is executed (if provided).

**Example**

```
score = 75

if score >= 90:
    print("Grade: A")
elif score >= 80:
    print("Grade: B")
elif score >= 70:
    print("Grade: C")
elif score >= 60:
    print("Grade: D")
else:
    print("Grade: F")
```

**Explanation:**

- The program checks if score >= 90.
    - Since 75 >= 90 is False, it moves to the next condition.

- Checks score >= 80.
    - 75 >= 80 is False.


Copyright © 2024 HyperionDev. All rights reserved.

- Checks score >= 70.
    - 75 >= 70 is True.
    - It executes print("Grade: C").
- The remaining elif and else statements are skipped.

**Output:**

```
Grade: C
```

**Key Points**

- **Order Matters**: Conditions are evaluated in the order they appear. Once a True condition is found, the rest are skipped.
- **Multiple elif Statements**: You can have as many elif statements as needed.
- **Optional else**: The else block is optional and executes if none of the if or elif conditions are True.
- **Single Execution**: Only one block of code among the if, elif, and else will be executed in a given pass.

**Using Logical Operators with elif**

You can use logical operators (and, or, not) to form complex conditions.

**Example:**

```
time = 14 # 14:00 hours

if time >= 5 and time < 12:
     print("Good morning!")
elif time >= 12 and time < 17:
    print("Good afternoon!")
elif time >= 17 and time < 21:
    print("Good evening!")
else:
    print("Good night!")
```


Copyright © 2024 HyperionDev. All rights reserved.

**Output:**

```
Good afternoon!
```

### Nested elif Statements

While nesting elif statements is uncommon, you can nest conditional statements within the blocks if needed.

**Example:**

```
age = 25

citizen = True
if age >= 18:
    if citizen:
        print("Eligible to vote.")
    else:
        print("Not a citizen.")
else:
    print("Too young to vote.")
```

### Important Notes

- **Indentation**: Consistent indentation is crucial for defining code blocks.
- **Colons**: Every if, elif, and else statement ends with a colon (:).
- **Boolean Conditions**: Conditions must evaluate to True or False.

### Common Use Cases

- **Grading Systems**: Assign grades based on score ranges.
- **Menu Selection**: Execute actions based on user menu choices.
- **Categorization**: Categorize data into different groups based on values.


Copyright © 2024 HyperionDev. All rights reserved.

## 1.5 For and while loops (KT0105) (IAC0101)

Loops are control structures that allow you to execute a block of code repeatedly. They are essential for performing tasks that require iteration, such as processing items in a list or executing code until a condition is met.

Python provides two primary types of loops:

1. **for Loops**

   **Purpose:**

   - **Iterate Over Sequences**: Execute a block of code for each item in a sequence (like lists, tuples, strings, or ranges).

2. **while Loops**

   **Purpose:**

   - **Execute Until Condition is False**: Repeat a block of code as long as a specified condition remains True.

**Key Differences**

- **for Loops**:
  - Used when you know the number of iterations or are iterating over a sequence.
  - Automatically handles iteration over items.

    Syntax:

    ```
    for variable in iterable:
        # Code to execute
    ```

    Here, the variable represents each item in the iterable, which can be a list, tuple, or range, and the loop iterates through these items one by one.

- **while Loops**:


Copyright © 2024 HyperionDev. All rights reserved.

- Used when the number of iterations is not known in advance.
- Continues until a condition becomes False.

Syntax:

```
while condition:
    # Code to execute
```

A while loop runs as long as a specified condition remains true. Its syntax is while condition: followed by the block of code to execute, where the condition is any logical test controlling the loop.

**Important Points**

- **Indentation**: Indentation defines the scope of the loop; consistent indentation is crucial.
- **Avoid Infinite Loops**: Ensure that the loop's condition will eventually become False to prevent infinite loops.
- **Control Statements**:
  - **break**: Exits the loop immediately.
  - **continue**: Skips the current iteration and continues with the next one.

## 1.6 Break, continue and pass statements (KT0106) (IAC0101)

In Python programming, break, continue, and pass statements are used to control the flow of loops and to manage how the program executes certain conditions.

**1. break Statement**

The break statement is used to exit or "break" out of a loop prematurely. When encountered within a loop (like a for or while loop), it immediately stops the loop execution and moves the control to the next statement after the loop.



Copyright © 2024 HyperionDev. All rights reserved.

**Example:**

```
for num in range(1, 10):
    if num == 5:
        break
    print(num)
```

*Output:*

```
1
2
3
4
```

In this example, the loop stops when num is equal to 5, so numbers after 4 are not printed.

**2. continue Statement**

The continue statement is used to skip the current iteration of the loop and move to the next iteration. When encountered, it stops the current loop body and proceeds to the next iteration without executing further statements in the loop body for that particular iteration.

**Example:**

```
for num in range(1, 6):
    if num == 3:
        continue
    print(num)
```

*Output:*

```
1
2
4
5
```



Copyright © 2024 HyperionDev. All rights reserved.

In this example, when num is 3, the continue statement is executed, so 3 is skipped, and the loop continues with the next number.

**3. pass Statement**

The pass statement is a null operation, meaning it does nothing. It acts as a placeholder where code is syntactically required but no action is needed. This is often used in places where code will be written later or for defining empty functions or classes.

**Example:**

```
for num in range(1, 4):
    if num == 2:
        pass # Placeholder for future code
    print(num)
```

*Output:*

```
1
2
3
```

In this example, when num is 2, pass does nothing, and the loop simply continues to print each number.



## Formative Assessment Activity 1

Complete the formative activity in your **Learner Workbook**, as per the instructions from your facilitator.



Copyright © 2024 HyperionDev. All rights reserved.

# Knowledge Topic 2

| Topic Code | KM-03-KT02: |
|---|---|
| Topic | Operators |
| Weight | 30% |

This knowledge topic will cover the following topic elements:

- Concept, definition and functions (KT0201)
- Syntax (KT0202)
- Types and uses (KT0203)
    - Arithmetic Operators
    - Relational/comparison Operators
    - Assignment Operators
    - Logical/ Bitwise Operators
    - Membership Operators
    - Identity Operators
    - Operator precedence

After working through this knowledge topic, your competence will be assessed according to the following criterion/criteria:

- IAC0201 Definitions, functions and features of Python operators are understood and explained



Copyright © 2024 HyperionDev. All rights reserved.

## 2.1 Concept, definition and functions (KT0201) (IAC0201)

Operators are special symbols or keywords in Python that perform operations on variables and values. They are essential for constructing expressions and performing calculations or manipulations within your programs.

Python provides a rich set of operators categorized into several groups:

1. Arithmetic Operators
2. Comparison (Relational) Operators
3. Assignment Operators
4. Logical Operators
5. Bitwise Operators
6. Membership Operators
7. Identity Operators

### 1. Arithmetic Operators

Used for mathematical calculations.

| Operator | Description | Example | Result |
|---|---|---|---|
| + | Addition | 5 + 3 | 8 |
| - | Subtraction | 5 - 3 | 2 |
| * | Multiplication | 5 * 3 | 15 |
| / | Division | 5 / 2 | 2.5 |
| % | Modulus (Remainder) | 5 % 2 | 1 |
| ** | Exponentiation | 5 ** 2 | 25 |
| // | Floor Division | 5 // 2 | 2 |


Copyright © 2024 HyperionDev. All rights reserved.

### 2. Comparison (Relational) Operators

Compare values and return a Boolean result (True or False).

| Operator | Description | Example | Result |
|---|---|---|---|
| == | Equal to | 5 == 3 | False |
| != | Not equal to | 5 != 3 | True |
| > | Greater than | 5 > 3 | True |
| < | Less than | 5 < 3 | False |
| >= | Greater than or equal to | 5 >= 5 | True |
| <= | Less than or equal to | 5 <= 3 | False |

### 3. Assignment Operators

Assign values to variables, possibly modifying them.

| Operator | Description | Example | Equivalent To |
|---|---|---|---|
| = | Assignment | a = 5 | 5 |
| += | Add and assign | a += 3 | a = a + 3 |
| -= | Subtract and assign | a -= 2 | a = a - 2 |
| *= | Multiply and assign | a *= 3 | a = a * 3 |
| /= | Divide and assign | a /= 2 | a = a / 2 |
| %= | Modulus and assign | a %= 3 | a = a % 3 |



Copyright © 2024 HyperionDev. All rights reserved.

| Operator | Description | Example | Equivalent To |
|---|---|---|---|
| **= | Exponentiate and assign | a **= 2 | a = a ** 2 |
| //= | Floor divide and assign | a //= 2 | a = a // 2 |

### 4. Logical Operators

Used to combine conditional statements.

| Operator | Description | Example | Result |
|---|---|---|---|
| And | Logical AND | (5 > 3) and (5 < 10) | True |
| Or | Logical OR | (5 > 3) or (5 > 10) | True |
| Not | Logical NOT | not(5 > 3) | False |

### 5. Bitwise Operators

Perform operations on binary representations of integers.

| Operator | Description | Example | Result |
|---|---|---|---|
| & | Bitwise AND | 5 & 3 | 1 |
| ` | ` | Bitwise OR | `5 |
| ^ | Bitwise XOR | 5 ^ 3 | 6 |
| ~ | Bitwise NOT | ~5 | -6 |
| << | Left Shift | 5 << 1 | 10 |


Copyright © 2024 HyperionDev. All rights reserved.

| Operator | Description | Example | Result |
|---|---|---|---|
| >> | Right Shift | 5 >> 1 | 2 |

### 6. Membership Operators

Test for membership in a sequence (such as strings, lists, or tuples).

| Operator | Description | Example | Result |
|---|---|---|---|
| In | True if value is found | 'a' in 'apple' | True |
| not in | True if value is not found | 'b' not in 'apple' | True |

### 7. Identity Operators

Compare the memory locations of two objects.

| Operator | Description | Example | Result |
|---|---|---|---|
| is | True if same object | a is b | Depends on a and b |
| is not | True if not same object | a is not b | Depends on a and b |



Copyright © 2024 HyperionDev. All rights reserved.

**Example:**

```
a = [1, 2, 3]
b = a
c = [1, 2, 3]
print(a is b)
# True, since b references the same object as a
print(a is c)
# False, since c is a different object with the same content
```

### Operator Precedence

When multiple operators are used in an expression, Python follows a specific order of operations, known as operator precedence. For example, multiplication and division have higher precedence than addition and subtraction.

**Example:**

```
result = 5 + 3 * 2 # Evaluates to 5+ (3 * 2) = 11
```

Use parentheses to override the default precedence:

```
result = (5 + 3) * 2 # Evaluates to (8) * 2 = 16
```

Operators are fundamental components in Python that allow you to perform calculations, compare values, assign variables, and build complex expressions. Understanding how each operator works and when to use it is crucial for effective programming in Python.

## 2.2 Syntax (KT0202) (IAC0201)

In the context of operators, **syntax** refers to the correct way to write operators in code, including the order, spacing, and placement of operands (the values on which operators act). Proper syntax is crucial for ensuring that operators function as expected.

Incorrect syntax, like using == instead of = in assignment, can lead to syntax errors or unexpected behaviour.


Copyright © 2024 HyperionDev. All rights reserved.

Operators in Python are special symbols or keywords that perform operations on operands (values or variables).

### 1. Arithmetic Operators

Perform mathematical calculations.

- Addition (+):
- Subtraction (-):
- Multiplication (*):
- Division (/):
- Floor Division (//):
- Modulus (%):
- Exponentiation (**):

### 2. Comparison (Relational) Operators

Compare values and return a Boolean result (True or False).

- Equal to (==):
- Not equal to (!=):
- Greater than (>):
- Less than (<):
- Greater than or equal to (>=):
- Less than or equal to (<=):

### 3. Assignment Operators

Assign values to variables, optionally performing an operation first.

- Simple assignment (=):
- Add and assign (+=):
- Subtract and assign (-=):
- Multiply and assign (*=):
- Divide and assign (/=):
- Floor divide and assign (//=):


Copyright © 2024 HyperionDev. All rights reserved.

- Modulus and assign (%=):
- Exponentiate and assign (**=):

**4. Logical Operators**

Combine conditional statements.

- Logical AND (and):
- Logical OR (or):
- Logical NOT (not):

**5. Membership Operators**

Test for membership in a sequence (like lists, tuples, or strings).

- In (in):
- Not In (not in):

**6. Identity Operators**

Check if two variables reference the same object.

- Is (is):
- Is Not (is not):

**7. Bitwise Operators**

Operate on binary representations of integers.

- Bitwise AND (&):
- Bitwise OR (|):
- Bitwise XOR (^):
- Bitwise NOT (~):
- Left Shift (<<):
- Right Shift (>>):



Copyright © 2024 HyperionDev. All rights reserved.

**Operator Precedence**

When multiple operators are used in an expression, Python evaluates them in a specific order known as operator precedence.

- **Highest Precedence to Lowest:**
    1. **Parentheses (()):** Expressions inside parentheses are evaluated first.
    2. **Exponentiation (**):**
    3. **Unary Plus and Minus (+, -), Bitwise NOT (~):**
    4. **Multiplication (*), Division (/), Floor Division (//), Modulus (%):**
    5. **Addition (+), Subtraction (-):**
    6. **Bitwise Shift Operators (<<, >>):**
    7. **Bitwise AND (&):**
    8. **Bitwise XOR (^):**
    9. **Bitwise OR (|):**
    10. **Comparison Operators (==, !=, >, <, >=, <=):**
    11. **Identity (is, is not) and Membership (in, not in) Operators:**
    12. **Logical NOT (not):**
    13. **Logical AND (and):**
    14. **Logical OR (or):**
    15. **Assignment Operators (=, +=, -= etc.):**

## 2.3 Types and uses (KT0203) (IAC0201)

In Python, **operators** are symbols or keywords that perform specific operations on one or more values, known as operands. Operators form the core of any programming language as they allow developers to manipulate data, make comparisons, and control the flow of the program. Python offers a wide variety of operators, each serving different purposes.



Copyright © 2024 HyperionDev. All rights reserved.

### 1. Arithmetic Operators

**Purpose:** Perform mathematical calculations.

**Operators and Uses:**

- **Addition (+):** Adds two operands.
    - *Example:* result = a + b adds a and b.
- **Subtraction (-):** Subtracts the second operand from the first.
    - *Example:* result = a - b subtracts b from a.
- **Multiplication (*):** Multiplies two operands.
    - *Example:* result = a * b multiplies a by b.
- **Division (/):** Divides the first operand by the second (returns a float).
    - *Example:* result = a / b divides a by b.
- **Floor Division (//):** Divides and returns the integer part of the quotient.
    - *Example:* result = a // b performs integer division.
- **Modulus (%):** Returns the remainder of the division.
    - *Example:* result = a % b gives the remainder of a divided by b.
- **Exponentiation (**):** Raises the first operand to the power of the second.
    - *Example:* result = a ** b calculates a raised to the power of b.

**Use Cases:**

- **Calculations:** Perform arithmetic computations.
- **Data Analysis:** Calculate sums, averages, and other statistical measures.
- **Algorithm Implementation:** Implement mathematical formulas and algorithms.

### 2. Comparison (Relational) Operators

**Purpose:** Compare values and return a Boolean result (True or False).

**Operators and Uses:**



Copyright © 2024 HyperionDev. All rights reserved.

- **Equal to (==):** Checks if two operands are equal.
    - *Example:* a == b returns True if a equals b.
- **Not equal to (!=):** Checks if two operands are not equal.
    - *Example:* a != b returns True if a is not equal to b.
- **Greater than (>):** Checks if the left operand is greater than the right.
    - *Example:* a > b returns True if a is greater than b.
- **Less than (<):** Checks if the left operand is less than the right.
    - *Example:* a < b returns True if a is less than b.
- **Greater than or equal to (>=):** Checks if the left operand is greater than or equal to the right.
    - *Example:* a >= b returns True if a is greater than or equal to b.
- **Less than or equal to (<=):** Checks if the left operand is less than or equal to the right.
    - *Example:* a <= b returns True if a is less than or equal to b.

**Use Cases:**

- **Decision Making:** Control program flow using if, elif, and while statements.
- **Filtering Data:** Select elements based on specific conditions.
- **Validation:** Ensure input data meets required criteria.

**3. Assignment Operators**

**Purpose:** Assign values to variables, potentially modifying them.

**Operators and Uses:**

- **Simple Assignment (=):** Assigns a value to a variable.
    - *Example:* a = b assigns the value of b to a.


Copyright © 2024 HyperionDev. All rights reserved.

- **Add and assign (+=):** Adds the right operand to the left operand and assigns the result to the left operand.
    - *Example:* a += b is equivalent to a = a + b.
- **Subtract and assign (-=):** Subtracts the right operand from the left operand and assigns the result.
    - *Example:* a -= b is equivalent to a = a - b.
- **Multiply and assign (*=):** Multiplies the left operand by the right operand and assigns the result.
    - *Example:* a *= b is equivalent to a = a * b.
- **Divide and assign (/=):** Divides the left operand by the right operand and assigns the result.
    - *Example:* a /= b is equivalent to a = a / b.
- **Modulus and assign (%=):** Calculates the modulus and assigns the result.
    - *Example:* a %= b is equivalent to a = a % b.
- **Exponentiate and assign (**=):** Raises the left operand to the power of the right operand and assigns the result.
    - *Example:* a **= b is equivalent to a = a ** b.
- **Floor divide and assign (//=):** Performs floor division and assigns the result.
    - *Example:* a //= b is equivalent to a = a // b.

**Use Cases:**

- **Updating Variables:** Efficiently modify variable values within loops or functions.
- **Counters and Accumulators:** Increment or decrement values in iterative processes.
- **Code Simplification:** Make code more concise and readable.


Copyright © 2024 HyperionDev. All rights reserved.

**4. Logical Operators**

**Purpose:** Combine conditional statements and return Boolean results.

**Operators and Uses:**

- **Logical AND (and):** Returns True if both operands are True.
    - *Example:* condition1 and condition2
- **Logical OR (or):** Returns True if at least one operand is True.
    - *Example:* condition1 or condition2
- **Logical NOT (not):** Returns True if the operand is False.
    - *Example:* not condition

**Use Cases:**

- **Complex Conditions:** Combine multiple conditions in control flow statements.
- **Decision Trees:** Implement multi-level decision-making processes.
- **Input Validation:** Ensure multiple criteria are met before proceeding.

**5. Bitwise Operators**

**Purpose:** Perform bit-level operations on integer types.

**Operators and Uses:**

- **Bitwise AND (&):** Performs a logical AND on each bit.
    - *Example:* a & b
- **Bitwise OR (|):** Performs a logical OR on each bit.
    - *Example:* a | b
- **Bitwise XOR (^):** Performs a logical XOR on each bit.
    - *Example:* a ^ b
- **Bitwise NOT (~):** Inverts all bits.
    - *Example:* ~a



Copyright © 2024 HyperionDev. All rights reserved.

- **Left Shift (<<):** Shifts bits to the left, filling with zeros.
    - *Example:* a << n shifts a left by n bits.
- **Right Shift (>>):** Shifts bits to the right, discarding bits.
    - *Example:* a >> n shifts a right by n bits.

**Use Cases:**

- **Low-Level Programming:** Optimize performance or memory usage.
- **Cryptography:** Implement encryption and decryption algorithms.
- **Graphics Programming:** Manipulate pixel data at the bit level.

**6. Membership Operators**

**Purpose:** Test for membership within a sequence, such as strings, lists, or tuples.

**Operators and Uses:**

- **In (in):** Returns True if a value is found in the sequence.
    - *Example:* 'apple' in fruits checks if 'apple' is in the fruits list.
- **Not In (not in):** Returns True if a value is not found in the sequence.
    - *Example:* 'banana' not in fruits checks if 'banana' is not in the fruits list.

**Use Cases:**

- **Search Operations:** Determine if an element exists within a collection.
- **Conditional Execution:** Execute code blocks based on membership.
- **Data Validation:** Check if inputs are within acceptable ranges or sets.

**7. Identity Operators**

**Purpose:** Compare the memory locations of two objects to determine if they are the same object.

**Operators and Uses:**


Copyright © 2024 HyperionDev. All rights reserved.

- **Is (is):** Returns True if both variables point to the same object.
    - *Example:* a is b
- **Is Not (is not):** Returns True if variables point to different objects.
    - *Example:* a is not b

**Use Cases:**

- **Object Comparison:** Check if two variables reference the same object in memory.
- **Singleton Checks:** Verify if a variable is None or another singleton.A **singleton** is a design pattern in programming that ensures only one instance of a particular class exists throughout the lifetime of an application. Think of it like a "one-of-a-kind" object that everyone shares and uses. This is useful when you want to control access to a shared resource, like a database connection or a configuration file.
    - *Example:* if variable is None:

Understanding the types and uses of operators in Python is fundamental for:

- **Performing Calculations:** Utilize arithmetic operators for mathematical operations.
- **Making Decisions:** Use comparison and logical operators in control flow statements.
- **Assigning and Updating Values:** Employ assignment operators to modify variables efficiently.
- **Manipulating Data Structures:** Leverage membership operators to work with collections.
- **Optimizing Code:** Apply bitwise and identity operators in specialized programming scenarios.


Copyright © 2024 HyperionDev. All rights reserved.

### 2.3.1 Arithmetic Operators

Arithmetic operators in Python are used to perform mathematical calculations such as addition, subtraction, multiplication, division, and more. They operate on numeric data types like integers (int) and floating-point numbers (float).

#### List of Arithmetic Operators

1. **Addition (+)**
    - **Purpose:** Adds two numbers.
    - **Syntax:** result = a + b
    - **Example:**

```
a = 5
b = 3
result = a + b # result is 8
```

2. **Subtraction (-)**
    - **Purpose:** Subtracts one number from another.
    - **Syntax:** result = a - b
    - **Example:**

```
a = 5
b = 3
result = a - b # result is 2
```

3. **Multiplication (*)**
    - **Purpose:** Multiplies two numbers.
    - **Syntax:** result = a * b
    - **Example:**


Copyright © 2024 HyperionDev. All rights reserved.

```
a = 5
b = 3
result = a a * b # result is 15
```

4. **Division (/)**
    - **Purpose:** Divides one number by another, returning a floating-point result.
    - **Syntax:** result = a / b
    - **Example:**

```
a = 57
b = 2
result = a / b # result is 2.5
```

5. **Floor Division (//)**
    - **Purpose:** Divides one number by another, returning the integer part of the quotient.
    - **Syntax:** result = a // b
    - **Example:**

```
a = 5
b = 2
result = a // b # result is 2
```

6. **Modulus (%)**
    - **Purpose:** Returns the remainder of the division between two numbers.
    - **Syntax:** result = a % b
    - **Example:**
    - **o**


Copyright © 2024 HyperionDev. All rights reserved.

```
a = 5
b = 2
result = a % b # result is 1
```

7. **Exponentiation (**)**
    - **Purpose:** Raises one number to the power of another.
    - **Syntax:** result = a ** b
    - **Example:**

```
a = 5
b = 3
result = a ** b # result is 125
```

**Usage Tips**

- **Order of Operations:** Python follows the standard mathematical order of operations (PEMDAS/BODMAS). Use parentheses () to control the order if needed.

```
result = (a + b) * c
```

- **Negative Numbers:** Arithmetic operators work with negative numbers as expected.

```
a = -5
b = 3
result =  a * b # result is -15
```

- **Type Conversion:** Be mindful of data types. Division (/) always returns a float. If you need an integer result, use floor division (//).

**Example Program**


Copyright © 2024 HyperionDev. All rights reserved.

```python
# Basic arithmetic operations
a = 10
b = 3
addition = a + b # 13
subtraction = a - b # 7


multiplication = a * b # 30
division = a / b # 3.333...
floor_division = a // b # 3
modulus = a % b # 1
exponentiation = a ** b # 1000

print("Addition: ", addition)
print("Subtraction:", subtraction)
print("Multiplication:", multiplication)
print("Division:", division)
print("Floor Division:", floor_division)
print("Modulus:", modulus)
print("Exponentiation:", exponentiation)
```

**Output:**

```
Addition: 13
Subtraction: 7
Multiplication: 30
Division: 3.3333333333333335
Floor Division: 3
Modulus: 1
Exponentiation: 1000
```

By using arithmetic operators, you can perform a wide range of mathematical calculations in your Python programs.

### 2.3.2 Relational/comparison Operators

Relational operators, also known as comparison operators, are used to compare two values or variables. They evaluate expressions and return a Boolean value: True if the


Copyright © 2024 HyperionDev. All rights reserved.

condition is met or False if it is not. These operators are fundamental in controlling the flow of a program by making decisions based on conditions.

**List of Relational Operators**

1. **Equal to (==)**
    - **Purpose:** Checks if the value of the left operand is equal to the value of the right operand.
    - **Syntax:** operand1 == operand2
    - **Example:**

```
a = 5
b = 5
result = a == b # result is True
```

2. **Not equal to (!=)**
    - **Purpose:** Checks if the value of the left operand is not equal to the value of the right operand.
    - **Syntax:** operand1 != operand2
    - **Example:**

```
a = 5
b = 3
result = a != b # result is True
```

3. **Greater than (>)**
    - **Purpose:** Checks if the value of the left operand is greater than the value of the right operand.
    - **Syntax:** operand1 > operand2
    - **Example:**


Copyright © 2024 HyperionDev. All rights reserved.

```
a = 5
b = 3
result = a > b # result is True
```

4. **Less than (<)**
    - **Purpose:** Checks if the value of the left operand is less than the value of the right operand.
    - **Syntax:** operand1 < operand2
    - **Example:**

```
a = 5
b = 7
result = a < b # result is True
```

5. **Greater than or equal to (>=)**
    - **Purpose:** Checks if the value of the left operand is greater than or equal to the value of the right operand.
    - **Syntax:** operand1 >= operand2
    - **Example:**

```
a = 5
b = 5
result = a >= b # result is True
```

6. **Less than or equal to (<=)**
    - **Purpose:** Checks if the value of the left operand is less than or equal to the value of the right operand.
    - **Syntax:** operand1 <= operand2
    - **Example:**


Copyright © 2024 HyperionDev. All rights reserved.

```
a = 5
b = 8
result = a <= b # result is True
```

### Usage in Conditional Statements

Relational operators are commonly used in conditional statements like if, elif, and while to control the flow of a program based on dynamic conditions.

### Example: Using Relational Operators in an if Statement

```
age = 18

if age >= 18:
    print("You are eligible to vote.")
else:
    print("You are not eligible to vote yet.")
```

### Explanation:

- The condition age >= 18 uses the greater than or equal to (>=) operator.
- If the condition is True, the program executes the code block under the if statement.
- If the condition is False, it executes the code under the else statement.

### Chaining Comparison Operators

Python allows you to chain comparison operators for more concise expressions.

### Example:

```
number = 15

if 10 < number < 20:
    print("The number is between 10 and 20.")
```

### Explanation:


Copyright © 2024 HyperionDev. All rights reserved.

- The expression 10 < number < 20 checks if the number is greater than 10 and less than 20.

**Using Relational Operators with Different Data Types**

Relational operators can also compare other data types, like strings and lists, based on lexicographical ordering or length.

**Example with Strings:**

```
str1 ="apple"
str2 = "banana"
result = str1 == str2 # result is False
result = str1 < str2 # result is True because "apple" comes before
"banana"
```

**Example with Lists:**

```
list1 = [1, 2, 3]
list2 = [1, 2, 4]

result = list1 == list2 # result is False
result = list1 < list2 # result is True
```

**Important Notes**

- **Boolean Results:** Relational operators always return a Boolean value (True or False).
- **Operator Precedence:** Relational operators have lower precedence than arithmetic operators but higher than logical operators.
- **Type Compatibility:** Be cautious when comparing different data types; incompatible types can lead to errors.


Copyright © 2024 HyperionDev. All rights reserved.

### Common Use Cases

- **Decision Making:** Control program flow with if, elif, and else statements.
- **Loops:** Use in loop conditions for while loops.
- **Validation:** Check if inputs meet certain criteria.
- **Sorting and Searching:** Compare elements when sorting data structures or searching for items.

## 2.3.3 Assignment Operators

Assignment operators are used in Python to assign values to variables. They can also perform operations on variables before assigning the result back to the same variable. These operators are essential for updating variable values efficiently and concisely.

### Basic Assignment Operator

1. **Simple Assignment (=)**
    - **Purpose:** Assigns the value on the right to the variable on the left.
    - **Syntax:**

```
variable = value
```

    - **Example:**

```
x  = 10
name = "Alice"
```

### Compound Assignment Operators

Compound assignment operators combine an arithmetic or bitwise operation with assignment, modifying the variable's value in place.

2. **Add and Assign (+=)**


Copyright © 2024 HyperionDev. All rights reserved.

- **Purpose:** Adds the right operand to the left operand and assigns the result to the left operand.
- **Syntax:**

```
variable += value  # Equivalent to variable = variable + value
```

- **Example:**

```
x = 5
x += 3 # x becomes 8
```

3. **Subtract and Assign (-=)**
   - **Purpose:** Subtracts the right operand from the left operand and assigns the result to the left operand.
   - **Syntax:**

```
variable -= value  # Equivalent to variable = variable - value
```

   - **Example:**

```
x = 5
x -= 2 # x becomes 3
```

4. **Multiply and Assign (*=)**
   - **Purpose:** Multiplies the left operand by the right operand and assigns the result to the left operand.
   - **Syntax:**

```
variable *= value  # Equivalent to variable = variable * value
```

   - **Example:**


Copyright © 2024 HyperionDev. All rights reserved.

```
x = 5
x *= 2 # x becomes 10
```

5. **Divide and Assign (/=)**
    - **Purpose:** Divides the left operand by the right operand and assigns the result to the left operand.
    - **Syntax:**

```
variable /= value  # Equivalent to variable = variable / value
```

    - **Example:**

```
x = 10
x /= 2 # x becomes 5.0
```

6. **Floor Divide and Assign (//=)**
    - **Purpose:** Performs floor division and assigns the integer result to the left operand.
    - **Syntax:**

```
variable //= value  # Equivalent to variable = variable // value
```

    - **Example:**

```
x = 10
x //= 3 # x becomes 3
```

7. **Modulus and Assign (%=)**
    - **Purpose:** Calculates the modulus and assigns the remainder to the left operand.
    - **Syntax:**


Copyright © 2024 HyperionDev. All rights reserved.

```
variable %= value  # Equivalent to variable = variable % value
```

- **Example:**

```
x = 10
x %= 3 # x becomes 1
```

8. **Exponentiate and Assign (**=)**
   - **Purpose:** Raises the left operand to the power of the right operand and assigns the result.
   - **Syntax:**

```
variable **= value  # Equivalent to variable = variable ** value
```

   - **Example:**

```
x = 2
x **= 3 # x becomes 8
```

9. **Bitwise AND and Assign (&=)**
   - **Purpose:** Performs bitwise AND operation and assigns the result.
   - **Syntax:**

```
variable &= value  # Equivalent to variable = variable & value
```

   - **Example:**

```
x = 5 #Binary: 0101
x &= 3 # x becomes 1 (Binary: 0001)
```


Copyright © 2024 HyperionDev. All rights reserved.

10. **Bitwise OR and Assign (|=)**
    - **Purpose:** Performs bitwise OR operation and assigns the result.
    - **Syntax:**

```
variable |= value  # Equivalent to variable = variable | value
```

    - **Example:**

```
x = 5 # Binary: 0101
x |= 3 # x becomes 7 (Binary: 0111)
```

11. **Bitwise XOR and Assign (^=)**
    - **Purpose:** Performs bitwise XOR operation and assigns the result.
    - **Syntax:**

```
variable ^= value  # Equivalent to variable = variable ^ value
```

    - **Example:**

```
x = 5 # Binary: 0101
x ^= 3 # x becomes 6 (Binary: 0110)
```

12. **Bitwise Right Shift and Assign (>>=)**
    - **Purpose:** Shifts the bits of the left operand right by the number of positions specified by the right operand and assigns the result.
    - **Syntax:**

```
variable >>= value  # Equivalent to variable = variable >> value
```

    - **Example:**


Copyright © 2024 HyperionDev. All rights reserved.

```
x = 8 # Binary: 1000
x >>= 2 # x becomes 2 (Binary: 0010)
```

13. **Bitwise Left Shift and Assign (<<=)**
    - **Purpose:** Shifts the bits of the left operand left by the number of positions specified by the right operand and assigns the result.
    - **Syntax:**

```
variable <<= value  # Equivalent to variable = variable << value
```

    - **Example:**

```
x = 3 # Binary: 0011
x <<= 2 # x becomes 12 (Binary: 1100)
```

### Usage Tips

- **Simplify Code:** Compound assignment operators make code more concise.

```
# Instead of:
total = total + amount

# Use:
total += amount
```

- **Data Types:** Be cautious with data types; division (/=) may convert integers to floats.
- **In-Place Modification:** Some objects, like lists, can be modified in place.

```
lst = [1, 2, 3]
lst  += [4, 5] # lst becomes [1, 2, 3, 4, 5]
```

### Common Use Cases

- **Counters and Accumulators:**


Copyright © 2024 HyperionDev. All rights reserved.

```
count = 0
for i in range(5):
    count += i # Sums up numbers from 0 to 4
```

- **Adjusting Values:**

```
temperature = 20
temperature -= 2 # Decrease temperature by 2 degrees
```

- **Bitwise Operations:**

```
flags = 0b1010
flags |= 0b0101 # Set specific bits
```

By understanding and utilizing assignment operators, you can write more efficient and readable Python code that updates variable values in a concise manner.

### 2.3.4 Logical/ Bitwise Operators

Logical operators are used to combine conditional statements and evaluate expressions that result in Boolean values (True or False). They are essential in controlling the flow of a program by enabling complex decision-making.

**List of Logical Operators**

1. **Logical AND (and)**
    - **Purpose:** Returns True if both operands are True; otherwise, returns False.
    - **Syntax:**

```
result = condition1 and condition2
```

    - **Example:**

```
a = 5
b = 10
result = (a > 0) and (b > 0) # result is True
```


Copyright © 2024 HyperionDev. All rights reserved.

2. **Logical OR (or)**
    - **Purpose:** Returns True if at least one of the operands is True; returns False only if both operands are False.
    - **Syntax:**

```
result = condition1 or condition2
```

    - **Example:**

```
a = -5
b = 10
result = (a > 0) or (b > 0) # result is True
```

3. **Logical NOT (not)**
    - **Purpose:** Inverts the Boolean value of the operand; converts True to False and vice versa.
    - **Syntax:**

```
result = not condition
```

    - **Example:**

```
a = 5
result = not (a > 0) # result is False
```

**Usage in Conditional Statements**

Logical operators are commonly used in if, elif, and while statements to create complex logical conditions.

**Example:**


Copyright © 2024 HyperionDev. All rights reserved.

```
age = 25
has_license = True

if age >= 18 and has_license:
    print("You can drive a car.")
else:
    print("You cannot drive a car.")
```

**Explanation:**

- The condition age >= 18 and has_license uses the logical AND operator.
- The code inside the if block executes only if both conditions are True.

### Bitwise Operators in Python

Bitwise operators perform operations on the binary representations of integers. They manipulate individual bits and are used in low-level programming, such as graphics programming, cryptography, and network programming.

### List of Bitwise Operators

1. **Bitwise AND (&)**
    - **Purpose:** Performs a logical AND operation on each pair of corresponding bits in two numbers.
    - **Syntax:**

```
result = operand1 & operand2
```

    - **Example:**


Copyright © 2024 HyperionDev. All rights reserved.

```
a = 5    # Binary: 0101
b = 3    # Binary: 0011
result = a & b  # result is 1 (Binary: 0001)
```

2. **Bitwise OR (|)**
    - **Purpose:** Performs a logical OR operation on each pair of corresponding bits.
    - **Syntax:**

```
result = operand1 | operand2
```

    - **Example:**

```
a = 5 # Binary: 0101
b = 3
result = a|b # result is 7 (Binary: 0111)
```

3. **Bitwise XOR (^)**
    - **Purpose:** Performs a logical XOR (exclusive OR) on each pair of corresponding bits.
    - **Syntax:**

```
result = operand1 ^ operand2
```

    - **Example:**

```
a = 5 # Binary: 0101
b = 3 # Binary: 0011
result = a^ b # result is 6 (Binary: 0110)
```

4. **Bitwise NOT (~)**


Copyright © 2024 HyperionDev. All rights reserved.

- **Purpose:** Inverts each bit of the operand (also known as one's complement).
- **Syntax:**

```
result = ~operand
```

- **Example:**

```
a = 5 # Binary: 0101
result = ~a # result is -6 (Binary: -(0101 + 1) = -0110)
```

5. **Left Shift (<<)**
   - **Purpose:** Shifts the bits of the number to the left by the specified number of positions; fills vacated bits with zeros.
   - **Syntax:**

```
result = operand << number_of_positions
```

   - **Example:**

```
a = 5 #Binary: 0101
result = a << 1 # result is 10 (Binary: 1010)
```

6. **Right Shift (>>)**
   - **Purpose:** Shifts the bits of the number to the right by the specified number of positions; for positive numbers, fills vacated bits with zeros.
   - **Syntax:**

```
result = operand >> number_of_positions
```

   - **Example:**


Copyright © 2024 HyperionDev. All rights reserved.

```
a = 5 # Binary: 0101
result = a >> 1 # result is 2 (Binary: 0010)
```

### Usage in Programming

Bitwise operators are often used in scenarios where performance is critical and memory usage needs to be optimized.

### Common Use Cases:

- **Setting, Clearing, and Toggling Bits:**

```
# Set a specific bit
flags = 0b0001
flags = 0b0010 # Sets the second bit: flags becomes Ob0011

# Clear a specific bit
flags &= ~0b0001 # Clears the first bit: flags becomes 0b0010

# Toggle a specific bit
flags ^= 0b0010 # Toggles the second bit: flags becomes 0b0000
```

- **Bit Masks:**

```
# Extract specific bits using a mask
value = 0b10101100
mask = 0b00001111
result = value & mask # result is 0b00001100
```

- **Efficient Multiplication or Division by Powers of Two:**

```
a = 4
result = a << 2 # Multiplies a by 2^2 (result is 16)
result = a >> 1 # Divides a by 2^1 (result is 2)
```


Copyright © 2024 HyperionDev. All rights reserved.

**Key Differences Between Logical and Bitwise Operators**

- **Operands:**
    - **Logical Operators:** Operate on Boolean expressions (True or False).
    - **Bitwise Operators:** Operate on integer values at the bit level.
- **Usage:**
    - **Logical Operators:** Used in control flow statements to make decisions based on conditions.
    - **Bitwise Operators:** Used for low-level programming tasks, such as manipulating individual bits in data structures.
- **Short-Circuit Behaviour (Logical Operators):**
    - In expressions using and or or, Python may skip evaluating the second operand if the first operand determines the result.
        - **Example:**

```
def func():
    print("Function called")
    return True

result = False and func() # func() is not called
```

Understanding both logical and bitwise operators is essential for different aspects of programming in Python:

- **Logical Operators:** Crucial for decision-making processes, controlling program flow, and evaluating complex conditions.
- **Bitwise Operators:** Important for performance-critical applications, low-level data manipulation, and tasks requiring direct bit manipulation.

### 2.3.5 Membership Operators

Membership operators are special operators that test for membership in a sequence (such as strings, lists, tuples, sets, or dictionaries). They evaluate to True or False based on whether a given value is present in the sequence.


Copyright © 2024 HyperionDev. All rights reserved.

### List of Membership Operators

1. **in Operator**
    - **Purpose:** Returns True if the specified value is found in the sequence.
    - **Syntax:**

```
result = value in sequence
```

    - **Example:**

```
fruits = ['apple', 'banana', 'cherry']
result = 'banana' in fruits # result is True
```

2. **not in Operator**
    - **Purpose:** Returns True if the specified value is **not** found in the sequence.
    - **Syntax:**

```
result = value not in sequence
```

    - **Example:**

```
fruits = ['apple', 'banana', 'cherry']
result = 'orange' not in fruits # result is True
```


Copyright © 2024 HyperionDev. All rights reserved.

### Usage with Different Data Types

Membership operators can be used with various data types:

- **Strings**

```
text = "Hello, world!"
result = 'world' in text # result is True
```

- **Lists**

```
numbers = [1, 2, 3, 4,5]
result = 3 in numbers # result is True
```

- **Tuples**

```
colors = ('red', 'green', 'blue')
result = 'yellow' not in colors # result is True
```

- **Sets**

```
vowels = {'a', 'e', 'i', 'o'}
result ='e' in vowels # result is True
```

- **Dictionaries**
    - When used with dictionaries, membership operators test for the presence of **keys**, not values.

```
person = {'name': 'Alice', 'age': 30}
result = 'name' in person # result is True
value_result = 'Alice' in person # result is False
```


Copyright © 2024 HyperionDev. All rights reserved.

## Practical Examples

1. **Checking User Input**

```
allowed_commands = ['start', 'stop', 'pause']
command = input("Enter a command: ")

if command in allowed_commands:
    print("Invalid command.")
else:
    print(f"Executing '{command}' command.")
```

2. **Filtering Data**

```
numbers = [1, 2, 3, 4, 5]
even_numbers = []

for num in numbers:
    if num % 2 == 0:
even_numbers.append(num)
    print("Even numbers:", even_numbers)
```

3. **Removing Unwanted Characters**

```
text = "Hello, World!"
unwanted_chars = ['!', “,”, “.”]

cleaned_text =
for char in text:
     if char not in unwanted_chars:
           cleaned_text += char
print("Cleaned text:", cleaned_text) # Output: "Hello World"
```


Copyright © 2024 HyperionDev. All rights reserved.

**Important Notes**

- **Case Sensitivity:** Membership tests are case-sensitive when dealing with strings.

```
text = "Python"
result = 'p' in text # result is False
```

- **Performance Considerations:** Membership tests on lists and tuples are linear-time operations (O(n)), while for sets and dictionaries, they are constant-time operations (O(1)).

**Common Use Cases**

- **Input Validation:** Check if user input matches acceptable values.
- **Data Filtering:** Include or exclude items from data structures.
- **Searching:** Determine the presence of an item before performing operations.

### 2.3.6 Identity Operators

Identity operators are used to compare the memory locations of two objects to determine whether they refer to the same object. They are essential when you need to check if two variables point to the **exact same object**, not just if they have equal values.

**List of Identity Operators**

1. **is Operator**
    - **Purpose:** Returns True if both variables point to the **same object** in memory.
    - **Syntax:**

```
result = object1 is object2
```

    - **Example:**

```
a = [1, 2, 3]
b = a
c = [1, 2, 3]
```


Copyright © 2024 HyperionDev. All rights reserved.

```
print(a is b) # True, because b references the same object as a
print(a is c) # False, because c is a different object with the same
content
```

2. **is not Operator**
    - **Purpose:** Returns True if both variables point to **different objects** in memory.
    - **Syntax:**

```
result = object1 is not object2
```

    - **Example:**

```
x = [4, 5, 6]
y = [4, 5, 6]
print(x is not y) # True, because x and y are different objects
```

**Understanding Object Identity**

- **Objects in Memory:** In Python, every object has a unique identifier, which can be obtained using the id() function. This identifier corresponds to the object's memory address.

```
a = [1, 2, 3]
b = a
c = [1, 2, 3]

print(id(a)) # e.g., 140505468713600
print(id(b)) # Same as id(a), because b references the same object
print(id(c)) # Different from id(a) and id(b)
```

- **Mutable vs. Immutable Objects:**
    - **Mutable Objects:** Objects like lists, dictionaries, and sets can change their content without changing their identity.


Copyright © 2024 HyperionDev. All rights reserved.

- **Immutable Objects:** Objects like integers, strings, and tuples cannot change their content once created.

**Difference Between is and ==**

- **is:** Checks whether two variables refer to the **same object** in memory.
- **==:** Checks whether the **values** of two objects are equal.

**Example:**

```
a = [1, 2, 3]
b = a
c = [1, 2, 3]

print(a == b) # True, values are equal
print(a is b) # True, same object
print(a == c) # True, values are equal
print(a is c) # False, different objects
```

**Common Use Cases**

1. **Checking for None**
    - It is a common practice to use is when comparing a variable to None.

```
result = None

if result is None:
    print("Result is None")
else:
    print("Result is not None")
```

    - **Why use is instead of ==?** Because None is a singleton in Python (there is only one instance of None), using is ensures you are checking for the exact None object.
2. **Ensuring Singleton Behaviour**
    - When implementing the Singleton design pattern, you can use is to verify that only one instance exists.


Copyright © 2024 HyperionDev. All rights reserved.

```
class Singleton:
    _instance = None

    def _new_(cls):
        if cls._instance is None:
           cls._instance = super(Singleton, cls).___new__(cls)
        return cls._instance

obj1 = Singleton()
obj2 = Singleton()
print(obj1 is obj2) # True, both are the same instance
```

The Singleton pattern is a way to make sure that only one instance of a class is created, even if you try to create multiple objects. In the code, _instance acts like a storage box that holds the single instance of the class. When you try to create a new object, the `__new__` method first checks if an instance already exists. If one doesn't exist, it creates a new one. If an instance is already there, it simply returns the existing one instead of making a new copy.

Think of it like having a single TV remote in a house. Even if multiple family members want to use a remote, they all have to use the same one, rather than each person getting a separate remote.

3. **Optimizing Comparisons**
    - For certain types of comparisons where object identity is more efficient than value equality (e.g., large data structures), is can be used.

**Important Notes**

- **Avoid Using is for Value Comparisons**
    - Using is for comparing values can lead to unexpected results, especially with immutable types like integers and strings due to Python's internal optimizations (like interning).
    - **Example:**

```
a = 256
b = 256
print(a is b) # True, because Python caches small integers

a = 257
b = 257
```


Copyright © 2024 HyperionDev. All rights reserved.

```
print(a is b) # Might be False, different objects
```

- **Best Practice**
    - Use == for value equality.
    - Use is for identity comparisons, especially with None.

```
if value == 10:
     print("Value is 10")
if value is None:
    print("Value is None")
```

### Understanding Interning

- **String Interning:**
    - Short strings and identifiers may be interned (cached) by Python, leading to is returning True.

```
s1 = "hello"
s2 = "hello"
print(s1 is s2) # True, due to interning
```

- **Integer Caching:**
    - Python caches small integers in the range -5 to 256.

```
a = 100
b = 100
print(a is b) # True

a = 1000
b = 1000
print(a is b) # False
```

### Practical Examples

1. **Comparing Custom Objects**


Copyright © 2024 HyperionDev. All rights reserved.

2. **Mutable Default Arguments Pitfall**
    - Using mutable default arguments can lead to unexpected behavior because the default argument is created only once, and all calls to the function share the same object.

```
def add_item(item, item_list=[]):
    item_list.append(item)
    return item_list

list1 = add_item(1)
list2 = add_item(2)

print(list1) # [1, 2]
print(list2) # [1, 2]
print(list1 is list2) # True
```

- **is Operator**
    - Checks if two variables refer to the **same object** in memory.
    - Use for identity comparisons, such as checking if a variable is None.
- **is not Operator**
    - Checks if two variables refer to **different objects**.
    - Useful for ensuring that a variable does not refer to a specific singleton object.
- **Key Differences from ==**
    - == checks for **value equality**.
    - is checks for **identity (same object in memory)**.
- **Best Practices**
    - Use is and is not for checking identity (e.g., None).
    - Use == and != for comparing values.


Copyright © 2024 HyperionDev. All rights reserved.

### 2.3.7 Operator precedence

Operator precedence determines the order in which operations are evaluated in expressions that contain multiple operators. When you write a complex expression with several operators, Python follows a specific set of rules to decide which operations to perform first.

**Order of Precedence**

Here is the general order of operator precedence in Python, from highest precedence to lowest:

1. **Parentheses ()**
    - **Purpose:** Expressions enclosed in parentheses are evaluated first.
    - **Example:**

```
result = (2 + 3) * 4 # result is 20
```

2. **Exponentiation ****
    - **Purpose:** Raises a number to the power of another.
    - **Example:**

```
result = 2 ** 3 ** 2 # result is 512 because it's evaluated as 2 ** (3 ** 2)
```

3. **Unary Plus, Unary Minus, Bitwise NOT +x, -x, ~x**
    - **Purpose:** Unary operations that affect a single operand.
    - **Example:**

```
result = -3 +5 # result is 2
```

4. **Multiplication *, Division /, Floor Division //, Modulus %**
    - **Purpose:** Perform multiplication and division operations.
    - **Example:**

```
result = 10 / 2 * 3 # result is 15.0
```

5. **Addition + and Subtraction -**
    - **Purpose:** Perform addition and subtraction.
    - **Example:**


Copyright © 2024 HyperionDev. All rights reserved.

```
result = 5 + 3 - 2 # result is 6
```

6. **Bitwise Shift Operators <<, >>**
    - **Purpose:** Shift bits left or right.
    - **Example:**

```
result = 1 << 2 # result is 4
```

7. **Bitwise AND &**
    - **Example:**

```
result = 5 & 3 # result is 1
```

8. **Bitwise XOR ^**
    - **Example:**

```
result = 5^ 3 # result is 6
```

9. **Bitwise OR |**
    - **Example:**

```
result = 5 | 3 # result is 7
```

10. **Comparisons ==, !=, >, <, >=, <=, is, is not, in, not in**
    - **Purpose:** Evaluate relationships between values.
    - **Example:**

```
result = 5 > 3 # result is True
```

11. **Logical NOT not**
    - **Purpose:** Inverts a Boolean value.
    - **Example:**

```
result = not True # result is False
```

12. **Logical AND and**
    - **Purpose:** Returns True if both operands are True.
    -


Copyright © 2024 HyperionDev. All rights reserved.

- **Example:**

```
result = True and False # result is False
```

13. **Logical OR or**
    - **Purpose:** Returns True if at least one operand is True.
    - **Example:**

```
result = False or True # result is True
```

14. **Conditional Expression if ... else**
    - **Purpose:** Ternary operator for conditional expressions.
    - **Example:**

```
result = 'Yes' if True else 'No' # result is 'Yes'
```

15. **Assignment Operators =, +=, -=, *=, /=, etc.**
    - **Purpose:** Assign values to variables.

    - **Example:**

```
x = 5
x += 3 # x is now 8
```

16. **Expression Lists (Comma-separated expressions)**
    - **Purpose:** Used in function calls, tuple unpacking, etc.
    - **Example:**

```
a, b = 1, 2
```

**Associativity**

- **Left-to-Right Associativity:** Most operators are left-associative, meaning expressions are evaluated from left to right.
    - **Example:**

```
result = 10 - 5 - 2 # Evaluated as (10 - 5) - 2, result is 3
```


Copyright © 2024 HyperionDev. All rights reserved.

- **Right-to-Left Associativity:** Some operators like exponentiation (**) are right-associative.
    - **Example:**

```
result = 2 ** 3 ** 2 # Evaluated as 2 ** (3 ** 2), result is 512
```

### Using Parentheses to Control Evaluation

Parentheses () can be used to override the default operator precedence and force certain parts of an expression to be evaluated first.

**Example:**

```
result = 5 + 3 * 2 # Without parentheses, result is 11
result (5 + 3) * 2 # With parentheses, result is 16
```

### Practical Examples

1. **Complex Expression:**

```
x = 5
y = 10
z = 15
result = x + y * z # Multiplication has higher precedence
# result is 5 + (10 * 15) = 155
```

2. **Logical Operations:**

```
a = True
b = False
c = True
result = a or b and c # 'and' has higher precedence than 'or'
# Evaluated as a or (b and c)
# result is True
```

To change the evaluation order:

```
result = (a or b) and c
# Evaluated as (True or False) and True
# result is True
```


Copyright © 2024 HyperionDev. All rights reserved.

3. **Chained Comparisons:**

Python allows for chaining comparisons, which can be more concise and readable.

```
x = 5
result = 1 < x < 10 # Equivalent to (1 < x) and (x < 10)
# result is True
```

- **Operator Precedence:** Defines the order in which operators are evaluated in an expression.
- **Higher Precedence Operators:** Are evaluated before lower precedence ones.
- **Associativity:** Determines the order of evaluation when operators have the same precedence.
- **Use Parentheses:** To make expressions clear and ensure they are evaluated as intended.



## Formative Assessment Activity 2

Complete the formative activity in your **Learner Workbook**, as per the instructions from your facilitator.



Copyright © 2024 HyperionDev. All rights reserved.

# Knowledge Topic 3

| Topic Code | KM-03-KT03: |
|---|---|
| Topic | Functions in Python |
| Weight | 15% |

This knowledge topic will cover the following topic elements:

- Concept, definition and functions (KT0301)
- Syntax (KT0302)
- Components (KT0303)
- Types of functions (KT0304)
- Built-in modules (KT0305)
- Main function and method: call, indentation, arguments and return values (KT0306)

After working through this knowledge topic, your competence will be assessed according to the following criterion/criteria:

- IAC0301 Definitions, functions and features of Python functions are understood and explained



Copyright © 2024 HyperionDev. All rights reserved.

## 3.1 Concept, definition and functions (KT0301) (IAC0301)

A **function** in Python is a reusable block of code that performs a specific task. Functions allow you to organise your code into manageable, logical sections, enhancing readability and maintainability. By encapsulating code within functions, you avoid repetition and facilitate code reuse.

### Defining a Function

In Python, you define a function using the def keyword, followed by the function name and parentheses (). Parameters (inputs) can be placed inside the parentheses. The function body contains the code to execute and is indented to indicate its scope.

**Syntax:**

```
def function_name(parameters):
    # Function body
    # Optional return statement
```

**Example:**

```
def greet (name):
    return f"Hello, {name}!"
```

### Uses and Benefits of Functions

- **Modularity:** Break down complex programs into smaller, manageable pieces.
- **Reusability:** Write code once and reuse it multiple times throughout your program.
- **Abstraction:** Hide complex implementation details behind simple function calls.
- **Maintainability:** Easier to update and debug code isolated within functions.
- **Readability:** Well-named functions make code more understandable and organised.
- **Testing:** Functions can be tested individually, improving reliability.



Copyright © 2024 HyperionDev. All rights reserved.

### Types of Functions

- **User-Defined Functions:** Created by programmers to perform specific tasks.
- **Built-in Functions:** Provided by Python, such as print(), len(), and input().
- **Anonymous Functions (Lambda Functions):** Short, one-line functions defined using the lambda keyword.

### Parameters and Arguments

- **Parameters:** Variables listed in a function's definition to accept inputs.
- **Arguments:** Actual values passed to the function when it is called.
- **Default Parameters:** Parameters with default values, making them optional.
- **Variable-Length Parameters:** Use *args for multiple positional arguments and **kwargs for multiple keyword arguments.

### Return Statement

- Functions can return a value using the return keyword.
- If no return statement is used, the function returns None by default.

### Scope of Variables

- **Local Scope:** Variables defined within a function, not accessible outside.
- **Global Scope:** Variables defined outside all functions, accessible throughout the code.



Copyright © 2024 HyperionDev. All rights reserved.

## 3.2 Syntax (KT0302) (IAC0301)

syntax in Python refers to the correct way to define, call, and structure functions so the interpreter can understand and execute them as intended. Functions allow you to encapsulate reusable blocks of code, making programs more modular and organised. Functions in Python are reusable blocks of code that perform a specific task. Defining and using functions helps to organise code, promote reusability, and make programs more readable.

### Defining a Function

To define a function in Python, you use the def keyword, followed by the function name, parentheses (), and a colon :. The function body contains indented statements that execute when the function is called.

**Syntax:**

```
def function_name(parameters):
    # Function body
    # Optional return statement
```

- **def**: Keyword used to define a function.
- **function_name**: The name you assign to the function (should follow naming conventions).
- **parameters**: Optional variables to accept input values (arguments).
- **Function Body**: Indented block of code that performs the function's operations.
- **return**: Optional statement to return a value from the function.

**Example:**

```
def greet (name):
    message  = f"Hello, {name}!"
    return message
```



Copyright © 2024 HyperionDev. All rights reserved.

**Function Components Explained**

1. **Function Name:**
    - Should be descriptive and written in lowercase letters with words separated by underscores (snake_case).

    **Example:** calculate_total, process_data.

2. **Parameters (Optional):**
    - Variables listed inside the parentheses.
    - Allow you to pass data into the function.
    - Can be zero or more parameters.

    **Example:**

```
def add(a, b):
    return a b
```

3. **Colon (:):**
    - Indicates the start of the function body.
    - Required at the end of the function definition line.
4. **Function Body:**
    - Contains the code that executes when the function is called.
    - Must be indented (usually by four spaces) to define the scope.
    - Can include statements, expressions, and other function calls.
5. **return Statement (Optional):**
    - Used to exit a function and optionally pass an expression back to the caller.
    - If omitted, the function returns None by default.


Copyright © 2024 HyperionDev. All rights reserved.

**Example:**

```
def square(number):
    return number * number
```

### Calling a Function

To execute a function, you call it by its name followed by parentheses, supplying arguments if needed.

**Syntax:**

```
function_name(arguments)
```

**Example:**

```
result = greet("Alice")
print(result) # Output: Hello, Alice!
```

### Default Parameters

You can provide default values for parameters, making them optional when calling the function.

**Syntax:**

```
def function_name(parameter=default_value):
    # Function body
```

**Example:**

```
def greet(name="Guest"):
    return f"Hello, {name}!"

print(greet()) # Output: Hello, Guest!
print(greet("Bob")) # Output: Hello, Bob!
```


Copyright © 2024 HyperionDev. All rights reserved.

### Variable-Length Arguments

- ***args**: Allows the function to accept any number of positional arguments, which are accessible as a tuple.

```
def sum_numbers(*args):
    return sum(args)
```

print(sum_numbers(1, 2, 3)) # Output: 6

```
# *kwargs** Allows the function to accept any number of keyword arguments

def print_info(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

print_info(name="Alice", age=30)
# Output:
# name: Alice
# age: 30
```

### Anonymous Functions (Lambda Functions)

For simple, one-time use functions, you can use the lambda keyword to create anonymous functions.

**Syntax:**

```
lambda parameters: expression
```

**Example:**

```
square = lambda x: x ** 2
print(square(5)) # Output: 25
```

### Docstrings (Optional)

You can include a documentation string at the beginning of the function body to describe what the function does.


Copyright © 2024 HyperionDev. All rights reserved.

**Syntax:**

```
def function_name(parameters):
    """Docstring explaining the function."""
    # Function body
```

**Example:**

```
def greet(name):
    '''Return a greeting message.'''
     return f"Hello, {name}!"
```

Accessing the docstring:

```
print(greet.__doc__) # Output: Return a greeting message.
```

**Key Points to Remember**

- **Indentation Matters:** The function body must be indented consistently.
- **Function Names:** Should be descriptive and follow naming conventions.
- **Parameters vs. Arguments:**
    - **Parameters:** Variables listed in the function definition.
    - **Arguments:** Values passed to the function when calling it.
- **Scope:** Variables defined inside a function are local to that function.
- **return Statement:**
    - Ends function execution and optionally returns a value.
    - Without it, the function returns None by default.

## 3.3 Components (KT0303) (IAC0301)

Functions in Python are reusable blocks of code that perform specific tasks. Understanding the components of a function is essential for defining and using them effectively. Below are the key components of functions in Python:


Copyright © 2024 HyperionDev. All rights reserved.

### 1. Function Definition (def Keyword)

- **Purpose:** Indicates the start of a function definition.
- **Syntax:** Begins with the keyword def.

**Example:**

```
def function_name():
    # Function body
```

### 2. Function Name

- **Purpose:** Identifies the function; used to call the function.
- **Naming Conventions:**
    - Should be descriptive and meaningful.
    - Use lowercase letters and underscores (snake_case).
    - Avoid using Python reserved keywords.

**Example:**

```
def calculate_total():
# Function body
```

### 3. Parameters (Arguments)

- **Purpose:** Variables that accept input values when the function is called.
- **Enclosed in parentheses () after the function name.
- **Types of Parameters:**
    - **Positional Parameters:** Required arguments in the correct order.
    - **Default Parameters:** Have default values and are optional.
    - **Variable-Length Parameters:**
        - *args: For multiple positional arguments (tuple).


Copyright © 2024 HyperionDev. All rights reserved.

- **kwargs: For multiple keyword arguments (dictionary).

**Example:**

```
def greet(name, message="Hello"):
    print(f" {message}, {name}!")
```

**4. Colon (:)**

- **Purpose:** Indicates the end of the function header and the beginning of the function body.
- **Syntax:** Placed after the parameter list.

**5. Function Body**

- **Purpose:** Contains the code that executes when the function is called.
- **Indentation:** Must be indented (usually by four spaces) to define the scope.
- **Can include:**
    - Statements and expressions.
    - Control flow statements (if, for, while).
    - Other function calls.

**Example:**

```
def add(a, b):
    result = a + b
    return result
```

**6. return Statement (Optional)**

- **Purpose:** Exits the function and optionally returns a value to the caller.
- **Syntax:** return expression
- **Behaviour:**
    - If omitted, the function returns None by default.
    - Can return multiple values as a tuple.


Copyright © 2024 HyperionDev. All rights reserved.

**Example:**

```
def square(number):
    return number * number
```

**7. Docstring (Documentation String) (Optional)**

- **Purpose:** Provides a description of the function's purpose and behaviour.
- **Syntax:** Placed as the first statement in the function body, enclosed in triple quotes """.
- **Accessed via:** The __doc__ attribute or help() function.

**8. Function Annotations (Optional)**

- **Purpose:** Provide type hints for parameters and return values.
- **Syntax:** Uses colons : after parameter names and -> before the return type.
- **Note:** They do not enforce types but serve as hints.

**Example:**

```
def square(number):
    return number * number
```

**9. Decorators (Optional, Advanced)**

- **Purpose:** Modify or enhance functions without changing their code.
- **Syntax:** Placed above the function definition with the @ symbol followed by the decorator name.

**Example:**

```
@decorator_function
def my_function():
    pass
```

**Putting It All Together: An Example**


Copyright © 2024 HyperionDev. All rights reserved.

```
def calculate_area(width: float, height: float) -> float:
    '"Calculate the area of a rectangle

     Parameters:
     width (float): The width of the rectangle.
     height (float): The height of the rectangle.

     Returns:
     float: The area of the rectangle.
     area = width * height "'

     return area
```

**Components Highlighted:**

- **def Keyword:** Starts the function definition.
- **Function Name:** calculate_area
- **Parameters with Annotations:** width: float, height: float
- **Colon (:):** Ends the function header.
- **Docstring:** Describes the function's purpose, parameters, and return value.
- **Function Body:** Calculates the area and assigns it to area.
- **return Statement:** Returns the computed area.
- **Return Type Annotation:** -> float

**Key Points to Remember**

- **Function Header:** Consists of the def keyword, function name, parameter list, and colon.
- **Function Body:**
    - Indented block of code under the function header.
    - Contains the executable code.
- **Parameters vs. Arguments:**


Copyright © 2024 HyperionDev. All rights reserved.

- **Parameters:** Variables listed in the function definition.
- **Arguments:** Actual values passed when calling the function.

- **Default Parameters:**
  - Provide default values for parameters, making them optional.
  - Must be specified after positional parameters.

**Example:**

```
def greet(name, message="Hello"):
    print(f" {message}, {name}!")
```

- **Variable-Length Arguments:**
  - ***args:** Accepts multiple positional arguments as a tuple.
  - ****kwargs:** Accepts multiple keyword arguments as a dictionary.

**Example with *args:**

```
def sum_numbers(*args):
    total = sum(args)
    return total
```

**Example with **kwargs:**

```
def print_details(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")
```

- **return Statement:**
  - Exits the function.
  - Can return a value or multiple values.
  - If omitted, returns None.
- **Docstrings:**


Copyright © 2024 HyperionDev. All rights reserved.

- Use triple quotes """ to enclose.
- First statement in the function body.
- Provides helpful documentation.

- **Function Annotations:**
  - Optional type hints.
  - Do not enforce data types.
- **Indentation Matters:**
  - Consistent indentation is crucial.
  - Typically, four spaces per indentation level.

**Example of a Function with All Components**

```
def process_data(data: list, operation: str = "sum") -> float:
    '''Process a list of numbers using the specified operation.

       Parameters:
       data (list): A list of numerical values.
       operation (str): The operation to perform ('sum' or 'average').

       Returns:
       float: The result of the operation.'''

    if operation == "sum":
        result = sum(data)
    elif operation "average":
        result = sum(data) / len(data)
    else:
        raise ValueError("Invalid operation specified.")
    return result
```

**Usage:**

```
numbers = [10, 20, 30]
print(process_data(numbers)) # Output: 60.0 (default is 'sum')
```


Copyright © 2024 HyperionDev. All rights reserved.

```
print(process_data(numbers, operation="average")) # Output: 20.0
```

By understanding these components, you can effectively define and utilize functions in Python, leading to more organised, reusable, and maintainable code.

## 3.4 Types of functions (KT0304) (IAC0301)

Python supports several types of functions, each serving different purposes in programming. Understanding these types helps you write more efficient and organised code.

### 1. Built-in Functions

- **Description:** Functions that are pre-defined in Python and are always available.
- **Characteristics:**
    - No need to import any modules to use them.
    - Provide basic functionality.
- **Examples:**
    - print(): Displays output to the console.
    - len(): Returns the length of an object.
    - input(): Reads input from the user.
    - type(): Returns the type of an object.

### 2. User-Defined Functions

- **Description:** Functions that programmers create to perform specific tasks.
- **Characteristics:**
    - Defined using the def keyword.
    - Can accept parameters and return values.


Copyright © 2024 HyperionDev. All rights reserved.

### 3. Anonymous Functions (Lambda Functions)

- **Description:** Small, unnamed functions defined using the lambda keyword.
- **Characteristics:**
    - Used for short, simple operations.
    - Can have any number of arguments but only one expression.
    - Often used in higher-order functions like map(), filter(), and sorted().
- **Example:**

```
square = lambda x: x**2
print(square(5)) # Output: 25
```

### 4. Recursive Functions

- **Description:** Functions that call themselves within their definition.
- **Characteristics:**
    - Used to solve problems that can be broken down into similar subproblems.
    - Must have a base case to prevent infinite recursion.
- **Example:**

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)
```

### 5. Higher-Order Functions

- **Description:** Functions that take other functions as arguments or return them as results.
- **Characteristics:**


Copyright © 2024 HyperionDev. All rights reserved.

- Facilitate functional programming paradigms.
- Enhance code reusability and modularity.

- **Examples:**
  - **Using a function as an argument:**

```
def apply_function(func, value):
    return func(value)

def double(x):
    return * * 2

print(apply_function(double, 5)) # Output: 10
```

  - **Built-in higher-order functions:**
    - map(): Applies a function to all items in an iterable.
    - filter(): Filters items in an iterable based on a function.

**6. Generator Functions**

- **Description:** Functions that yield a sequence of values using the yield keyword instead of return.
- **Characteristics:**
  - Generate values on the fly and maintain state between iterations.
  - More memory-efficient than returning a list of all values.
- **Example:**

```
def fibonacci(n):
    a, b = 0, 1
    while a < n:
        yield a
        a, b = b, a + b

for num in fibonacci(10):
print(num)
```


Copyright © 2024 HyperionDev. All rights reserved.

### 7. Built-in Methods

- **Description:** Functions associated with objects and data types in Python.
- **Characteristics:**
    - Called on an object using dot notation.
    - Perform operations related to the object's data type.
- **Examples:**
    - **String Methods:**

```
text = "hello"
print(text.upper()) # Output: HELLO
```

    - **List Methods:**

```
numbers = [1, 2, 3]
numbers.append(4)
print(numbers) # Output: [1, 2, 3, 4]
```

### 8. Partial Functions

- **Description:** Functions created from existing functions with some arguments fixed.
- **Characteristics:**
    - Useful for customizing functions with fixed parameters.
    - Created using the functools.partial() function.
- **Example:**

```
from functools import partial

def multiply(a, b):
    return a * b
```


Copyright © 2024 HyperionDev. All rights reserved.

```
double = partial (multiply, b=2)
print(double(5)) # Output: 10
```

### 9. Decorators

- **Description:** Functions that modify the behaviour of other functions.
- **Characteristics:**
    - Use the @decorator_name syntax above a function definition.
    - Can add functionality to existing functions without changing their code.
- **Example:**

```
def uppercase_decorator (func):
    def wrapper():
        result = func()
        return result.upper()
    return wrapper

@uppercase_decorator
def greet():
    return "hello"

print(greet()) # Output: HELLO
```

### 10. Coroutine Functions

- **Description:** Functions used for asynchronous programming, defined with async def.
- **Characteristics:**
    - Can pause and resume execution using await.
    - Used with the asyncio library for concurrent code.


Copyright © 2024 HyperionDev. All rights reserved.

- **Example:**

```
import asyncio

async def say_hello():
    print("Hello")
    await asyncio.sleep(1)
    print("World")

asyncio.run(say_hello())
```

Understanding the different types of functions in Python allows you to:

- Write more efficient and readable code.
- Choose the appropriate function type for a given task.
- Leverage Python's features for functional and asynchronous programming.

## 3.5 Built-in modules (KT0305) (IAC0301)

Python comes with a rich standard library of built-in modules that provide a wide range of functionalities. These modules are part of the Python Standard Library and are included with your Python installation. They help you perform various tasks without the need to install external packages.

### Purpose of Built-in Modules

- **Code Reusability:** Offer pre-written code for common tasks.
- **Efficiency:** Optimize performance for standard operations.
- **Convenience:** Simplify complex tasks with easy-to-use interfaces.

### Commonly Used Built-in Modules

#### 1. sys Module

- **Purpose:** Provides access to some variables and functions that interact with the Python interpreter.


Copyright © 2024 HyperionDev. All rights reserved.

- **Common Uses:**
    - Access command-line arguments (sys.argv).
    - Exit the program (sys.exit()).
    - Get the Python version (sys.version).

**Example:**

```
import sys
print("Python version:", sys.version)
```

**2. os Module**

- **Purpose:** Provides functions for interacting with the operating system.
- **Common Uses:**
    - Work with file paths (os.path).
    - Create or remove directories (os.mkdir(), os.rmdir()).
    - Get environment variables (os.environ).

**Example:**

```
import os
print("Current working directory:", os.getcwd())
```

**3. math Module**

- **Purpose:** Offers mathematical functions defined by the C standard.
- **Common Uses:**
    - Perform advanced mathematical calculations (e.g., trigonometry, logarithms).
    - Use constants like math.pi and math.e.


Copyright © 2024 HyperionDev. All rights reserved.

**Example:**

```
import math
print("Square root of 16:", math.sqrt(16))
print("Value of pi:", math.pi)
```

### 4. random Module

- **Purpose:** Implements pseudo-random number generators.
- **Common Uses:**
    - Generate random numbers (random.randint(), random.random()).
    - Shuffle sequences (random.shuffle()).
    - Choose random elements (random.choice()).

**Example:**

```
import random
numbers = [1, 2, 3, 4, 5]
random.shuffle (numbers)
print("Shuffled list:", numbers)
```

### 5. datetime Module

- **Purpose:** Supplies classes for manipulating dates and times.
- **Common Uses:**
    - Get the current date and time (datetime.now()).
    - Format dates and times.
    - Calculate time differences (timedelta objects).

**Example:**

```
from datetime import datetime
now = datetime.now()
print("Current date and time:", now.strftime("%Y-%m-%d %H:%M:%S"))
```


Copyright © 2024 HyperionDev. All rights reserved.

### 6. re Module (Regular Expressions)

- **Purpose:** Provides support for regular expressions.
- **Common Uses:**
    - Search, match, and manipulate strings using patterns.
    - Validate input formats (e.g., email addresses, phone numbers).

**Example:**

```
import re
pattern = r'\d+'
text = "There are 12 apples"
match = re.search(pattern, text)
if match:
    print("Found number:", match.group())
```

### 7. json Module

- **Purpose:** Enables working with JSON data.
- **Common Uses:**
    - Parse JSON strings into Python objects (json.loads()).
    - Convert Python objects to JSON strings (json.dumps()).
    - Read from and write to JSON files.

**Example:**

```
import json
data = {'name': 'Alice', 'age': 30}
json_str=json.dumps (data)
print("JSON string:", json_str)
```

### 8. csv Module

- **Purpose:** Facilitates CSV file reading and writing.
- **Common Uses:**
    - Read data from CSV files (csv.reader()).


Copyright © 2024 HyperionDev. All rights reserved.

- Write data to CSV files (csv.writer()).

**Example:**

```
import csv
with open('data.csv', newline='') as csvfile:
    reader = csv.reader(csvfile)
    for row in reader:
        print(row)
```

### 9. urllib Module

- **Purpose:** Provides functions for working with URLs.
- **Common Uses:**
  - Fetch data from the web (urllib.request).
  - Parse URLs (urllib.parse).

**Example:**

```
from urllib.request import urlopen
response = urlopen('http://example.com')
html = response.read()
print("HTML content length:", len(html))
```

### 10. collections Module

- **Purpose:** Implements specialized container datatypes.
- **Common Uses:**
  - Use deque for efficient queue operations.
  - Use Counter for counting hashable objects.
  - Use OrderedDict to remember the order entries were added.

**Example:**

```
from collections import Counter
data = ['apple', 'banana', 'apple', 'orange', 'banana', 'apple']
counts = Counter(data)
```


Copyright © 2024 HyperionDev. All rights reserved.

```
print("Counts:", counts)
```

### 11. itertools Module

- **Purpose:** Offers functions creating iterators for efficient looping.
- **Common Uses:**
    - Generate permutations and combinations.
    - Create infinite iterators.

**Example:**

```
import itertools
for combination in itertools.combinations ([1, 2, 3], 2):
    print(combination)
```

### 12. functools Module

- **Purpose:** Provides higher-order functions and operations on callable objects.
- **Common Uses:**
    - Use partial to fix a certain number of arguments of a function.
    - Implement caching with lru_cache.
    - Use reduce to apply a rolling computation.

**Example:**

```
from functools import reduce
numbers = [1, 2, 3, 4]
product = reduce(lambda x, y: x * y, numbers)
print("Product of numbers:", product)
```

### 13. operator Module

- **Purpose:** Exports a set of efficient functions corresponding to the intrinsic operators.
- **Common Uses:**


Copyright © 2024 HyperionDev. All rights reserved.

- Replace lambdas for better performance and readability.
- Perform arithmetic and logical operations.

**Example:**

```
import operator
numbers = [1, 2, 3, 4]
result = operator.add(5, 3)
print("Addition result:", result)
```

### 14. copy Module

- **Purpose:** Provides functions for copying objects.
- **Common Uses:**
  - Create shallow (copy.copy()) or deep copies (copy.deepcopy()) of objects.

**Example:**

```
import copy
original = [1, [2, 3], 4]
shallow_copy = copy.copy(original)
deep_copy = copy.deepcopy (original)
```

### 15. logging Module

- **Purpose:** Enables logging events for tracking and debugging.
- **Common Uses:**
  - Record debug information.
  - Configure logging levels and outputs.

**Example:**

```
import logging
logging.basicConfig(level=logging.INFO)
logging.info("This is an info message")
```

### Key Points


Copyright © 2024 HyperionDev. All rights reserved.

- **Accessing Modules:** Use the import statement to include a module in your program.

```
import module_name
```

- **Selective Import:** Import specific functions or classes using the from keyword.

```
from module_name import function_name
```

- **Aliasing Modules:** Use the as keyword to give a module or function an alias.

```
import module_name as mn
```

- **Module Documentation:** You can access a module's documentation using the help() function or by reading the official Python documentation.

**Benefits of Using Built-in Modules**

- **Time-Saving:** Reduce development time by leveraging pre-built functionality.
- **Reliability:** Trusted and well-tested code maintained by the Python community.
- **Efficiency:** Optimized implementations for common tasks.
- **Portability:** Code using standard library modules is more portable across different environments.

Understanding and utilizing Python's built-in modules allows you to:

- **Enhance Productivity:** Focus on solving higher-level problems rather than reinventing the wheel.
- **Write Cleaner Code:** Simplify complex tasks with concise and readable code.
- **Improve Code Quality:** Rely on proven implementations to reduce bugs.


Copyright © 2024 HyperionDev. All rights reserved.

## 3.6 Main function and method: call, indentation, arguments and return values (KT0306) (IAC0301)

In Python, the concept of a "main" function is a convention used to indicate the starting point of a program. While Python does not have a built-in main() function like some other languages, you can define one and use the if __name__ == "__main__": construct to execute code only when the script is run directly.

**Defining and Using the Main Function:**

```
def main():
    # Main program logic goes here
    print("Hello, World!")

if name __main__":
    main()
```

**Explanation:**

- **def main():**: Defines a function named main.
- **Function Body**: Contains the code that runs when the function is called.
- **if __name__ == "__main__":**: Checks if the script is being run directly (not imported as a module). If true, it calls the main() function.

**Purpose:**

- Organises code and improves readability.
- Ensures that certain code runs only when the script is executed directly.

**2. Function and Method Calls**

**Function Calls**

A **function call** executes the code within a function. You call a function by writing its name followed by parentheses, including any required arguments.

**Syntax:**



Copyright © 2024 HyperionDev. All rights reserved.

```
function_name(arguments)
```

**Example:**

```
def main():
    # Main program logic goes here
    print("Hello, World!")

if name __main__":
    main()
```

### Method Calls

A **method** is a function associated with an object. Methods are called on objects using dot notation. In Object-Oriented Programming (OOP), objects are like real-life things that combine data (attributes) and actions (methods). Think of a car object—it can have attributes like color or model, and actions like drive or honk. Methods are the actions you can call on an object using dot notation.

**Syntax:**

```
object.method_name(arguments)
```

**Example:**

```
text = "hello"
print(text.upper()) # Output: HELLO
```

- **text.upper()**: Calls the upper method on the string object text, converting it to uppercase.

### 3. Indentation in Functions

**Indentation** is crucial in Python to define the scope of code blocks like functions, loops, and conditionals. Python uses indentation (spaces or tabs) to determine which statements belong together.

**Rules:**



Copyright © 2024 HyperionDev. All rights reserved.

- The standard indentation is **4 spaces**.
- All code inside a function must be indented at the same level.
- Misaligned indentation will result in a IndentationError.

**Example of Correct Indentation:**

```
def add(a, b):
    result = a + b
    return result
```

**Example of Incorrect Indentation:**

```
def add(a, b):
result = a + b
    return result
```

**Tips:**

- Be consistent with indentation throughout your code.
- Avoid mixing tabs and spaces.

**4. Function Arguments**

Functions can accept inputs called **arguments** or **parameters**. These allow functions to process data and produce dynamic results.

**Types of Arguments**

1. **Positional Arguments:**
    - Passed to functions in order.
    - **Example:**

```
def subtract(a, b):
    return a b

result = subtract(10, 5) # a=10, b=5
```

2. **Keyword Arguments:**
    - Passed to functions by explicitly naming each parameter.
    - Order doesn't matter.
    - **Example:**

```
result = subtract(b=5, a=10) # a=10, b=5
```


Copyright © 2024 HyperionDev. All rights reserved.

3. **Default Arguments:**
    - Parameters with default values if no argument is provided.
    - **Example:**

```
def greet(name, message="Hello"):
    print(f" {message}, {name}!")

greet("Bob") # Output: Hello, Bob!
greet("Alice", message="Hi") # Output: Hi, Alice!
```

4. **Variable-Length Arguments:**
    - ***args**: Accepts any number of positional arguments as a tuple.

```
def multiply(*args):
    result = 1
    for num in args:
        result *= num
    return result

print(multiply(2, 3, 4)) # Output: 24
```

    - ****kwargs**: Accepts any number of keyword arguments as a dictionary.

```
def display_info(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

display_info(name="Alice", age=30)
# Output:
# name: Alice
# age: 30
```

### 5. Return Values of Functions

Functions can send back data to the caller using the **return** statement.

#### Using Return Statements

- **Single Value Return:**


Copyright © 2024 HyperionDev. All rights reserved.

```
def square(x):
    return x * x

result = square(5) # result is 25
```

- **Multiple Values Return:**

Functions can return multiple values as a tuple.

```
def get_coordinates():
    x = 10
    y = 20
    return x, y

coord = get_coordinates() # coord is (10, 20)
x_coord, y_coord = get_coordinates()
print(x_coord) # Output: 10
print(y_coord) # Output: 20
```

- **No Return Statement:**

If no return statement is used, the function returns None by default.

```
def say_hello():
    print("Hello!")

result = say_hello() # Prints "Hello!"
print(result) # Output: None
```

**Purpose of Return Values**

- **Data Processing:** Functions can process input data and return results.
- **Flow Control:** The return statement exits the function, optionally sending back data.
- **Reusability:** Returned values can be used elsewhere in the code.

**6. Putting It All Together: Example**


Copyright © 2024 HyperionDev. All rights reserved.

```
def main():
    # Get user input
    name = input("Enter your name: ")
    # Call the greet function and print the returned message
     print(greet (name))

def greet (name, message="Welcome"):
    '''Generate a greeting message.'''
    return f"{message}, {name}!"

if __name__ == __main__:
     main()
```

**Explanation:**

- **Main Function (main()):**
    - Serves as the entry point of the program.
    - Gets user input and calls other functions.
- **Function Call:**
    - greet(name) calls the greet function with the argument name.
- **Indentation:**
    - All code inside main() and greet() is properly indented.
- **Arguments:**
    - greet accepts a positional argument name and a default argument message.
- **Return Value:**
    - greet returns a greeting string, which is then printed in main().
- **Program Execution Control:**
    - if __name__ == "__main__": ensures main() runs only when the script is executed directly.

- **Main Function:**
    - Not mandatory in Python but useful for organising code.
    - Use if __name__ == "__main__": to execute code when the script is run directly.


Copyright © 2024 HyperionDev. All rights reserved.

- **Function and Method Calls:**
    - **Functions:** Called by their name followed by parentheses and arguments.
    - **Methods:** Called on objects using dot notation.
- **Indentation:**
    - Essential for defining code blocks.
    - Consistent indentation (usually 4 spaces) is required.
- **Arguments:**
    - **Positional Arguments:** Passed in order.
    - **Keyword Arguments:** Passed with parameter names.
    - **Default Arguments:** Have default values.
    - **Variable-Length Arguments:** Use *args and **kwargs.
- **Return Values:**
    - Use return to send back data.
    - Can return multiple values as tuples.
    - Without return, functions return None.

By understanding these concepts, you can effectively define and use functions in Python, leading to cleaner, more modular, and maintainable code.



## Formative Assessment Activity 3

Complete the formative activity in your **Learner Workbook**, as per the instructions from your facilitator.



Copyright © 2024 HyperionDev. All rights reserved.

# Knowledge Topic 4

| | |
|---|---|
| **Topic Code** | KM-03-KT04: |
| **Topic** | Date, Time and Calendar |
| **Weight** | 15% |

This knowledge topic will cover the following topic elements:

- Concept, definition and functions (KT0401)
- Syntax (KT0402)
- Dates, times and time intervals (KT0403)
- Datetime module (KT0404)
- Classes (KT0405)
- Timedelta objects (KT0406)

After working through this knowledge topic, your competence will be assessed according to the following criterion/criteria:

- IAC0401 Definitions, functions and features of Python date, time and calendar are understood and explained



Copyright © 2024 HyperionDev. All rights reserved.

## 4.1 Concept, definition and functions (KT0401) (IAC0401)

Python provides built-in modules to work with dates, times, and calendars, enabling developers to handle temporal data efficiently. These modules allow for creating date and time objects, performing arithmetic operations, formatting and parsing dates and times, and interacting with calendar information.

**Key Modules and Their Functions**

1. **datetime Module**
    - **Purpose:** Manipulate dates and times.
    - **Key Classes and Functions:**
        - **datetime.datetime**: Represents both date and time.
        - **datetime.date**: Represents a date (year, month, day).
        - **datetime.time**: Represents time independent of the date.
        - **datetime.timedelta**: Represents the difference between two dates or times.
        - **datetime.strptime()**: Parses a string into a datetime object.
        - **datetime.strftime()**: Formats a datetime object into a string.
2. **time Module**
    - **Purpose:** Provides time-related functions based on Unix timestamps.
    - **Key Functions:**
        - **time.time()**: Returns the current time in seconds since the epoch.


Copyright © 2024 HyperionDev. All rights reserved.

- **time.sleep(secs)**: Suspends execution for the given number of seconds.
- **time.ctime(secs)**: Converts a time expressed in seconds to a readable string.

3. **calendar Module**
    - **Purpose:** Offers functions related to calendars.
    - **Key Functions:**
        - **calendar.month(year, month)**: Returns a string representing the month's calendar.
        - **calendar.isleap(year)**: Determines if a year is a leap year.
        - **calendar.weekday(year, month, day)**: Returns the day of the week for a given date.

**Common Operations and Examples**

- **Creating Dates and Times:**

```
from datetime import datetime, date, time

now = datetime.now() # Current date and time
today = date.today() # Current date
specific_time = time(14, 30) # Time at 14:30
```

- **Date Arithmetic:**

```
from datetime import timedelta

tomorrow = today + timedelta(days=1) # Adding one day
duration = timedelta(hours=2, minutes=30) # Duration of 2.5 hours
```


Copyright © 2024 HyperionDev. All rights reserved.

- **Formatting and Parsing Dates:**

```
formatted_date = now.strftime("%Y-%m-%d %H:%M:%S") # Format to string
parsed_date = datetime.strptime("2023-10-05", "%Y-%m-%d") # Parse from
string
```

- **Working with Calendars:**

```
import calendar
print(calendar.month (2023, 10)) # Display October 2023 calendar
is_leap = calendar.isleap(2024) # Check if 2024 is a leap year
day_of_week = calendar.weekday(2023, 10, 5) # Get the weekday of a date
```

**Time Zones (Python 3.9+):**

- **Using zoneinfo Module:**

```
from datetime import datetime
from zoneinfo import ZoneInfo
utc_now = datetime.now(tz-ZoneInfo("UTC")) # Current UTC time
ny_time = utc_now.astimezone (ZoneInfo("America/New_York")) # Convert to
New York tim
```

Python's built-in modules for date, time, and calendar provide essential tools for handling temporal data:

- **datetime** for creating and manipulating date and time objects.
- **time** for working with Unix timestamps and measuring time intervals.
- **calendar** for calendar-related functions like generating calendars and checking leap years.

These modules are crucial for applications involving scheduling, logging, time calculations, and localization.


Copyright © 2024 HyperionDev. All rights reserved.

## 4.2 Syntax (KT0402) (IAC0401)

Python provides built-in modules to handle date, time, and calendar operations:

- **datetime Module**: For creating and manipulating date and time objects.
- **time Module**: For time-related functions based on Unix timestamps.
- **calendar Module**: For calendar-related functions.

### 1. datetime Module Syntax

#### Importing the Module:

```
from datetime import datetime, date, time, timedelta
```

#### Creating Date and Time Objects:

- **Current Date and Time:**

```
now = datetime.now()
```

- **Specific Date:**

```
specific_date = date(year, month, day)
# Example:
specific_date = date(2023, 10, 5)
```

- **Specific Time:**

```
specific_time = time (hour, minute, second)
# Example:
specific_time = time(14, 30, 0)
```

#### Formatting and Parsing Dates:

- **Formatting (strftime):**


Copyright © 2024 HyperionDev. All rights reserved.

```
formatted_date = datetime_object.strftime(format_string)
# Example:
formatted_date = now.strftime("%Y-%m-%d %H:%M:%S")
```

- **Parsing (strptime):**

```
datetime_object = datetime.strptime(date_string, format_string)
# Example:
date_string = "2023-10-05 14:30:00"
parsed_date = datetime.strptime(date_string, "%Y-%m-%d %H:%M:%S")
```

**Date Arithmetic with timedelta:**

- **Creating a Time Delta:**

```
delta = timedelta (days=1, hours=2)
```

- **Adding/Subtracting Time Delta:**

```
future_date = datetime_object + delta
past_date = datetime_object - delta
```

**2. time Module Syntax**

**Importing the Module:**

```
import time
```

**Getting Current Time:**

- **Epoch Time (Seconds since Jan 1, 1970):**

```
epoch_time = time.time()
```

**Pausing Execution:**

- **Sleep for a Number of Seconds:**

```
time.sleep(seconds)
```


Copyright © 2024 HyperionDev. All rights reserved.

```
# Example:
time.sleep(2) # Pauses execution for 2 seconds
```

### 3. calendar Module Syntax

#### Importing the Module:

```
import calendar
```

#### Displaying Calendars:

- **Monthly Calendar:**

```
print(calendar.month(year, month))
# Example:
print(calendar.month(2023, 10))
```

#### Checking for Leap Years:

- **Determine if a Year is a Leap Year:**

```
is_leap = calendar.isleap(year)
# Example:
is_leap = calendar.isleap(2024) # Returns True
```

#### Getting Weekday of a Date:

- **Get the Day of the Week (0=Monday):**

```
weekday = calendar.weekday (year, month, day)
# Example:
weekday = calendar.weekday(2023, 10, 5)
```

#### Common Format Codes for Date and Time

Used with strftime and strptime methods:

- %Y - Four-digit year (e.g., 2023)


Copyright © 2024 HyperionDev. All rights reserved.

- %m - Two-digit month (01-12)
- %d - Two-digit day of the month (01-31)
- %H - Hour in 24-hour format (00-23)
- %M - Minute (00-59)
- %S - Second (00-59)
- %A - Full weekday name (e.g., Monday)
- %a - Abbreviated weekday name (e.g., Mon)
- %B - Full month name (e.g., October)
- %b - Abbreviated month name (e.g., Oct)

- **Import Modules:**

```
from datetime import datetime, date, time, timedelta
import time
import calendar
```

- **Create Date and Time Objects:** Use constructors like datetime(), date(), and time().
- **Format and Parse Dates:** Use strftime() to format dates and times into strings, and strptime() to parse strings into date and time objects.
- **Perform Date Arithmetic:** Use timedelta for adding or subtracting time intervals.
- **Work with Calendars:** Use the calendar module to display calendars and check for leap years.

## 4.3 Dates, times and time intervals (KT0403) (IAC0401)

Python provides robust support for working with dates and times through its built-in modules. The primary module for handling date and time data is the datetime module, which allows you to create, manipulate, and format date and time objects. Additionally, the time and calendar modules offer supplementary functionalities.


Copyright © 2024 HyperionDev. All rights reserved.

Working with **dates, times, and time intervals** is essential in many Python applications, from managing timestamps in data analysis to scheduling tasks and logging events in web applications. Python provides several libraries and modules to make handling dates and times straightforward and efficient.

**Key Concepts:**

1. **Dates**: Represent specific days (e.g., 2024-01-01). Python's datetime module allows you to create, manipulate, and format date objects easily.
2. **Times**: Represent a specific time of day (e.g., 14:30:00 for 2:30 PM). Time objects can be used for precise time tracking and manipulation within a given day.
3. **DateTime Objects**: These combine both date and time information, enabling more complex operations involving both components, such as handling time zones and calculating durations between specific dates and times.
4. **Time Intervals**: Known as "timedeltas" in Python, these represent differences between dates or times, such as the number of days between two dates or the time difference between two events.

**1. The datetime Module**

The datetime module supplies classes for manipulating dates and times in both simple and complex ways. Key classes include:

- **date**: Represents a calendar date (year, month, and day).
- **time**: Represents time independent of any particular day (hour, minute, second, microsecond).
- **datetime**: Combines date and time information into a single object.
- **timedelta**: Represents the difference between two dates or times, useful for date arithmetic.

**Example: Creating Date and Time Objects**


Copyright © 2024 HyperionDev. All rights reserved.

```python
from datetime import date, time, datetime, timedelta

# Current date
today = date.today()
print("Today's date:", today)

# Specific date
specific_date = date(2023, 10, 5)
print("Specific date:", specific_date)

# Current time
current_time = datetime.now().time()
print("Current time:", current_time)

# Specific time
specific_time = time(14, 30, 0)
print("Specific time:", specific_time)

# Current date and time
now = datetime.now()
print("Current datetime:", now)

# Specific datetime
specific_datetime = datetime(2023, 10, 5, 14, 30, 0)
print("Specific datetime:", specific_datetime)
```

### 2. Time Intervals with timedelta

The timedelta class represents a duration, which can be added to or subtracted from date or datetime objects.


Copyright © 2024 HyperionDev. All rights reserved.

**Example: Working with timedelta**

```
# Time interval of 2 days and 3 hours
time_interval = timedelta(days=2, hours=3)

# Adding time interval to a date
future_date = today + time_interval
print("Future date:", future_date)

# Difference between two dates
date1 = date(2023, 10, 5)
date2 = date(2023, 10, 10)
difference = date2 -  date1
print("Difference in days:", difference.days)
```

### 3. Formatting and Parsing Dates and Times

Use strftime to format date and time objects into readable strings and strptime to parse strings into date and time objects.

**Formatting with strftime:**

```
# Format datetime object to string
formatted_datetime = now.strftime("%Y-%m-%d %H:%M:%S")
print("Formatted datetime:", formatted_datetime)
```

**Parsing with strptime:**

```
# Parse string to datetime object
date_string = "2023-10-05 14:30:00"
parsed_datetime = datetime.strptime(date_string, "%Y-%m-%d %H:%M:%S")
print("Parsed datetime:", parsed_datetime)
```

**Common Format Codes:**

- %Y: Four-digit year (e.g., 2023)
- %m: Two-digit month (01-12)
- %d: Two-digit day of the month (01-31)
- %H: Hour in 24-hour format (00-23)
- %M: Minute (00-59)


Copyright © 2024 HyperionDev. All rights reserved.

- %S: Second (00-59)
- %A: Full weekday name (e.g., Monday)
- %B: Full month name (e.g., October)

### 4. Working with Time Zones

From Python 3.9 onwards, the zoneinfo module provides support for time zones.

**Example: Using Time Zones**

```
from datetime import datetime
from zoneinfo import ZoneInfo

# Current time in UTC
utc_now = datetime.now(tz-ZoneInfo("UTC"))
print("Current UTC time:", utc_now)

# Convert to another time zone
new_york_time = utc_now.astimezone (ZoneInfo("America/New_York"))
print("New York time:", new_york_time)
```

### 5. The time Module

The time module provides time-related functions, primarily based on Unix timestamps.

**Example: Using the time Module**

```
import time

# Current time in seconds since the epoch
epoch_time = time.time()
print("Time since epoch:", epoch_time)

# Convert epoch time to readable format
readable_time = time.ctime(epoch_time)
print("Readable time:", readable_time)

# Pause execution for 2 seconds
print("Pausing for 2 seconds...")
time.sleep(2)
print("Resumed execution.")
```


Copyright © 2024 HyperionDev. All rights reserved.

### 6. The calendar Module

The calendar module offers calendar-related functions.

#### Example: Using the calendar Module

```
import calendar

# Print the calendar for October 2023
print(calendar.month (2023, 10))

# Check if a year is a leap year
is_leap = calendar.isleap(2024)
print("Is 2024 a leap year?", is_leap)

# Get the weekday of a specific date (0=Monday)
weekday = calendar.weekday(2023, 10, 5)
print("Weekday of 2023-10-05:", weekday)
```

### 7. Date Arithmetic

You can perform arithmetic operations on date and time objects using timedelta.

#### Example: Date Arithmetic

```
# Subtracting dates
days_difference = (date2 - date1).days
print(f"There are {days_difference} days between {date1} and {date2}.")

# Adding days to a date
future_date = datel + timedelta(days=7)
print("Date after 7 days:", future_date)
```

### 8. Accessing Date and Time Components

You can extract specific components from date and time objects.


Copyright © 2024 HyperionDev. All rights reserved.

**Example: Extracting Components**

```
print("Year:" now. year)
print("Month:", now.month)
print("Day: ", now.day)
print("Hour:", now.hour)
print("Minute:", now.minute)
print("Second: ", now.second)
```

- **datetime Module**: Primary tool for working with dates and times.
    - **date**: For date objects (year, month, day).
    - **time**: For time objects (hour, minute, second).
    - **datetime**: Combines date and time.
    - **timedelta**: Represents time intervals.
- **Formatting and Parsing**: Use strftime and strptime for converting between date/time objects and strings.
- **Time Zones**: Handle time zones with the zoneinfo module (Python 3.9+).
- **time Module**: Provides functions for working with Unix timestamps and delays.
- **calendar Module**: Offers calendar-related functions like generating calendars and checking for leap years.
- **Date Arithmetic**: Perform calculations with dates and times using timedelta.

These tools allow you to effectively manage date and time data in Python, whether you're scheduling events, logging timestamps, or performing time-based calculations.


Copyright © 2024 HyperionDev. All rights reserved.

## 4.4 Datetime module (KT0404) (IAC0401)

The datetime module in Python provides classes for manipulating dates and times. It allows you to perform a wide range of operations on date and time data, such as creating specific dates or times, performing arithmetic, formatting, and parsing.

**Key Classes in the datetime Module**

1. **datetime.date**
    - **Purpose:** Represents a date (year, month, and day) without time information.
    - **Usage:**

```
from datetime import date
today = date.today()
specific_date = date(2023, 10, 5)
```

2. **datetime.time**
    - **Purpose:** Represents time (hour, minute, second, microsecond) without date information.
    - **Usage:**

```
from datetime import time
current_time = time(14, 30, 0)
```

3. **datetime.datetime**
    - **Purpose:** Combines date and time into a single object.
    - **Usage:**

```
from datetime import datetime
now = datetime.now()
specific_datetime = datetime(2023, 10, 5, 14, 30, 0)
```


Copyright © 2024 HyperionDev. All rights reserved.

4. **datetime.timedelta**
    - **Purpose:** Represents the difference between two date or time objects (duration).
    - **Usage:**

```
from datetime import timedelta
duration = timedelta (days=2, hours=3)
```

5. **datetime.timezone**
    - **Purpose:** Represents time zone information for datetime objects.
    - **Usage:**

```
from datetime import timezone, timedelta
utc = timezone.utc
offset = timezone (timedelta(hours=-5)) # For UTC-5
```

### Common Operations with datetime

1. **Creating Date and Time Objects**
    - **Current Date and Time:**

```
now = datetime.now()
today = date.today()
```

    - **Specific Date and Time:**

```
specific_date = date(2023, 10, 5)
specific_time = time(14, 30, 0)
specific_datetime = datetime(2023, 10, 5, 14, 30, 0)
```

2. **Date and Time Arithmetic**
    - **Adding or Subtracting Time using timedelta:**

```
future_date = today + timedelta(days=10)
```


Copyright © 2024 HyperionDev. All rights reserved.

```
past_datetime = now - timedelta (hours=2)
```

- **Calculating Duration between Dates:**

```
date1 = date(2023, 10, 5)
date2 = date(2023, 11, 5)
difference = date2 - date1 # Returns a timedelta object
days_between = difference.days
```

3. **Formatting Dates and Times**
   - **Using strftime to Format:**

```
formatted_date = now.strftime("%Y-%m-%d %H:%M:%S")
```

   - **Common Format Codes:**
     - %Y - Year with century (e.g., 2023)
     - %m - Month as a zero-padded decimal number (01-12)
     - %d - Day of the month (01-31)
     - %H - Hour in 24-hour format (00-23)
     - %M - Minute (00-59)
     - %S - Second (00-59)

4. **Parsing Dates and Times**
   - **Using strptime to Parse Strings into datetime Objects:**

```
date_string = "2023-10-05 14:30:00"
parsed_datetime = datetime.strptime(date_string, "%Y-%m-%d %H:%M:%S")
```

5. **Working with Time Zones**
   - **Using timezone Class:**


Copyright © 2024 HyperionDev. All rights reserved.

```
from datetime import timezone, timedelta
tz = timezone (timedelta(hours=-5)) # Time zone offset of -5 hours
localized_datetime = datetime(2023, 10, 5, 14, 30, 0, tzinfo=tz)
```

- **Using zoneinfo Module (Python 3.9+):**

```
from datetime import datetime
from zoneinfo import ZoneInfo
ny_time = datetime.now(tz=ZoneInfo("America/New_York"))
```

6. **Accessing Date and Time Components**

- **Extracting Attributes:**

```
year = now. year
month = now.month
day = now.day
hour = now.hour
minute = now.minute
second = now.second
```

**Example Usage:**

```
from datetime import datetime, timedelta

# Get current datetime
now = datetime.now()
print("Current datetime:", now)

# Format datetime
formatted = now.strftime("%Y-%m-%d %H:%M:%S")
print("Formatted datetime:", formatted)

# Parse datetime from string
date_string = "2023-10-05 14:30:00"
parsed_date = datetime.strptime(date_string, "%Y-%m-%d %H:%M:%S")
print("Parsed datetime:", parsed_date)

# Date arithmetic
future_date = now + timedelta(days=7)
print("Date after 7 days:", future_date)
```


Copyright © 2024 HyperionDev. All rights reserved.

```
# Difference between dates
difference = future_date - now
print("Difference in days:", difference.days)
```

**Why Use the datetime Module?**

- **Handling Date and Time Data:** Essential for applications that require date and time operations, such as logging, event scheduling, and data analysis.
- **Date Arithmetic:** Perform calculations like finding the difference between dates or adding time intervals.
- **Formatting and Parsing:** Convert date and time objects to strings and vice versa, which is crucial for displaying information to users or reading date/time data from strings.
- **Time Zone Support:** Work with dates and times across different time zones.

**Key Points to Remember**

- **Immutable Objects:** date, time, and datetime objects are immutable; their values cannot be changed after creation.
- **Time Zone Handling:** Be cautious when working with time zones to avoid errors related to daylight saving time and time zone conversions.
- **Locale Considerations:** The strftime and strptime methods are affected by the locale setting of the system.

The datetime module is a powerful tool in Python for working with dates and times. By mastering its classes and methods, you can handle a wide range of date and time-related tasks efficiently and effectively in your programs.


Copyright © 2024 HyperionDev. All rights reserved.

## 4.5 Classes (KT0405) (IAC0401)

Python offers robust support for handling dates, times, and calendars through various classes in its standard library modules. The primary modules are:

- **datetime Module**: Provides classes for manipulating dates and times.
- **calendar Module**: Offers classes and functions related to calendar operations.

### 1. datetime Module Classes

The datetime module supplies classes for manipulating dates and times in both simple and complex ways.

#### a. datetime.date Class

- **Purpose**: Represents a calendar date (year, month, and day) without time information.
- **Attributes**:
  - **year**: Four-digit year.
  - **month**: Month (1-12).
  - **day**: Day of the month (1-31).
- **Methods**:
  - **today()**: Returns the current local date.
  - **fromtimestamp(timestamp)**: Creates a date object from a POSIX timestamp.
  - **isoformat()**: Returns the date as a string in YYYY-MM-DD format.
- **Example**:

```
from datetime import date

# Current date
today = date.today()
print("Today's date:", today)

# Specific date
specific_date = date(2023, 10, 5)
print("Specific date:", specific_date)
```

#### b. datetime.time Class


Copyright © 2024 HyperionDev. All rights reserved.

- **Purpose**: Represents time (hour, minute, second, microsecond) independent of any particular day.
- **Attributes**:
    - **hour**: Hour (0-23).
    - **minute**: Minute (0-59).
    - **second**: Second (0-59).
    - **microsecond**: Microsecond (0-999999).
- **Methods**:
    - **isoformat()**: Returns the time as a string in HH:MM:SS format.
- **Example**:

```
from datetime import time
# Specific time
specific_time = time(14, 30, 0)
print("Specific time:", specific_time)
```

**c. datetime.datetime Class**

- **Purpose**: Combines date and time into a single object.
- **Attributes**:
    - All attributes from date and time classes.
- **Methods**:
    - **now()**: Returns the current local date and time.
    - **utcnow()**: Returns the current UTC date and time.
    - **fromtimestamp(timestamp)**: Creates a datetime object from a POSIX timestamp.
    - **strftime(format)**: Formats the datetime using a format string.
    - **strptime(date_string, format)**: Parses a string into a datetime object.


Copyright © 2024 HyperionDev. All rights reserved.

- **Example**:

```
from datetime import datetime

# Current datetime
now = datetime.now()
print("Current datetime:", now)

# Specific datetime
specific_datetime = datetime(2023, 10, 5, 14, 30, 0)
print("Specific datetime:", specific_datetime)
```

### d. datetime.timedelta Class

- **Purpose**: Represents a duration or difference between two dates or times.
- **Attributes**:
    - **days**, **seconds**, **microseconds**
- **Methods**:
    - Supports arithmetic operations with date and datetime objects.
- **Example**:

```
from datetime import timedelta

# Time interval of 2 days and 3 hours
time_interval = timedelta(days=2, hours=3)
```

### e. datetime.timezone Class

- **Purpose**: Represents a fixed offset from UTC (used in time zone calculations).
- **Attributes**:
    - **tzinfo**: Time zone information.
- **Methods**:
    - **utc**: Represents the UTC time zone.
- **Example**:


Copyright © 2024 HyperionDev. All rights reserved.

```
from datetime import datetime, timezone, timedelta

# UTC time zone
utc = timezone.utc
# Time zone with offset
offset = timezone (timedelta(hours=-5)) # UTC-5

# Datetime with time zone
localized_datetime = datetime(2023, 10, 5, 14, 30, 0, tzinfo=offset)
print("Localized datetime:", localized_datetime)
```

### 2. calendar Module Classes

The calendar module provides classes to output calendars and provide additional useful functions related to the calendar.

#### a. calendar.Calendar Class

- **Purpose**: Provides methods to generate data for calendars.
- **Methods**:
    - **itermonthdates(year, month)**: Iterates over dates in a month.
    - **monthdatescalendar(year, month)**: Returns a list of weeks in a month, each week is a list of date objects.
- **Example**:

```
import calendar

cal = calendar.Calendar()
for day in cal.itermonthdates (2023, 10):
    print(day)
```

#### b. calendar.TextCalendar Class

- **Purpose**: Generates plain text calendars.
- **Methods**:
    - **formatmonth(year, month)**: Returns a month's calendar as a multi-line string.


Copyright © 2024 HyperionDev. All rights reserved.

- **formatyear(year)**: Returns a year's calendar as a multi-line string.

- **Example**:

```
import calendar

text_cal = calendar.TextCalendar()
print(text_cal.formatmonth (2023, 10))
```

### c. calendar.HTMLCalendar Class

- **Purpose**: Generates HTML-formatted calendars.
- **Methods**:
  - **formatmonth(year, month)**: Returns a month's calendar as an HTML table.
  - **formatyear(year)**: Returns a year's calendar as an HTML table.
- **Example**:

```
import calendar

html_cal = calendar.HTMLCalendar()
html_calendar = html_cal.formatmonth(2023, 10)
print(html_calendar)
```

### d. calendar.LocaleTextCalendar and calendar.LocaleHTMLCalendar Classes

- **Purpose**: Generate calendars with locale-specific formatting.
- **Usage**:
  - Initialize with a locale setting to format month and weekday names appropriately.
- **Example**:

```
import calendar
locale_text_cal = calendar.LocaleTextCalendar (locale='fr_FR')
print(locale_text_cal.formatmonth (2023, 10))
```


Copyright © 2024 HyperionDev. All rights reserved.

### 3. Common Tasks Using These Classes

#### a. Date and Time Arithmetic

- **Adding/Subtracting Time Intervals**:

```
from datetime import datetime, timedelta
now = datetime.now()
future_time = now + timedelta(days=7)
print("Date after 7 days:", future_time)
```

- **Calculating Differences Between Dates**:

```
date1 = datetime (2023, 10, 5)
date2 = datetime (2023, 11, 5)

difference = date2 - date1
print("Difference:", difference)
```

#### b. Formatting and Parsing Dates

- **Formatting Dates**:

```
formatted_date = now.strftime("%Y-%m-%d %H:%M:%S")
print("Formatted date:", formatted_date)
```

- **Parsing Dates**:

```
from datetime import datetime

date_string = "2023-10-05 14:30:00"
parsed_date = datetime.strptime(date_string, "%Y-%m-%d %H:%M:%S")
print("Parsed date:", parsed_date)
```

#### c. Generating Calendars

- **Print a Monthly Calendar**:


Copyright © 2024 HyperionDev. All rights reserved.

```
import calendar
print(calendar.month (2023, 10))
```

- **Check for Leap Year**:

```
is_leap = calendar.isleap(2024)
print("Is 2024 a leap year?", is_leap)
```

### 4. Time Zone Handling

From Python 3.9 onwards, you can use the built-in zoneinfo module for more comprehensive time zone support.

**Using zoneinfo.ZoneInfo Class**

- **Purpose**: Represents a time zone as defined in the IANA time zone database.
- **Usage**:

```
from datetime import datetime
from zoneinfo import ZoneInfo

# Current time in UTC
utc_now = datetime.now(tz-ZoneInfo("UTC"))
print("Current UTC time:", utc_now)

# Convert to another time zone
ny_time = utc_now.astimezone(ZoneInfo("America/New_York"))
print("New York time:", ny_time)
```

- **datetime Module Classes**:
    - **date**: For working with dates (year, month, day).
    - **time**: For working with times (hour, minute, second, microsecond).
    - **datetime**: Combines date and time into one object.
    - **timedelta**: Represents durations for arithmetic operations.
    - **timezone**: For time zone handling.


Copyright © 2024 HyperionDev. All rights reserved.

- **calendar Module Classes**:
    - **Calendar**: Base class for calendar-related methods.
    - **TextCalendar**: Generates plain text calendars.
    - **HTMLCalendar**: Generates HTML-formatted calendars.
    - **LocaleTextCalendar** and **LocaleHTMLCalendar**: Locale-aware versions.
- **Common Operations**:
    - **Date Arithmetic**: Adding or subtracting timedelta objects.
    - **Formatting and Parsing**: Using strftime and strptime.
    - **Time Zone Handling**: Using timezone and zoneinfo modules.
    - **Generating Calendars**: Using classes from the calendar module.

**Practical Applications**

- **Scheduling and Deadlines**: Calculate future dates or times based on durations.
- **Logging and Timestamps**: Record the exact date and time of events.
- **Time Zone Conversions**: Convert times between different time zones.
- **User Interfaces**: Display calendars in applications or websites.
- **Data Analysis**: Process and analyse time-series data.

By understanding and utilizing these classes in Python, you can effectively manage and manipulate date, time, and calendar data in your applications. Whether you're building a scheduling app, processing historical data, or simply need to display the current date and time, these tools provide the functionality you need.

## 4.6 Timedelta objects (KT0406) (IAC0401)

The timedelta class in Python's datetime module represents a duration, the difference between two dates or times. It is used to perform arithmetic operations on date,


Copyright © 2024 HyperionDev. All rights reserved.

datetime, and time objects, allowing you to calculate time intervals, add or subtract time periods, and compare dates and times.

**Purpose of timedelta**

- **Duration Representation**: Encapsulates differences in times, expressed in days, seconds, and microseconds.
- **Date Arithmetic**: Enables addition and subtraction with date and datetime objects.
- **Time Calculations**: Useful for calculating future or past dates and times based on a time interval.

**Creating timedelta Objects**

You can create a timedelta object by specifying any of the following parameters:

- days
- seconds
- microseconds
- milliseconds
- minutes
- hours
- weeks

**Syntax:**

```
from datetime import timedelta

# Create a timedelta object
delta = timedelta(days=1, hours=2, minutes=30)
```

**Example:**

```
from datetime import timedelta
```



Copyright © 2024 HyperionDev. All rights reserved.

```
# Time interval of 2 days and 3 hours
time_interval = timedelta(days=2, hours=3)
print("Time interval:", time_interval)
```

**Output:**

```
Time interval: 2 days, 3:00:00
```

### Attributes of timedelta

timedelta objects have attributes that store the duration in normalized values:

- **days**: Number of days.
- **seconds**: Number of seconds (0 to 86399).
- **microseconds**: Number of microseconds (0 to 999999).

**Accessing Attributes:**

```
delta = timedelta(days=2, hours=3, minutes=15)
print("Days: ", delta.days)
print("Seconds:", delta.seconds)
print("Microseconds:", delta.microseconds)
```

**Output:**

```
Days: 2
Seconds: 11700
Microseconds: 0
```

*Note:* The hours and minutes are converted into seconds and added to the seconds attribute.

### Arithmetic Operations with timedelta

#### 1. Adding or Subtracting timedelta Objects

You can perform arithmetic operations between timedelta objects.

**Example:**

```
delta1 = timedelta(days=2)
delta2 = timedelta(hours=5)
```



Copyright © 2024 HyperionDev. All rights reserved.

```
# Addition
total_delta = delta1 + delta2
print("Total Duration:", total_delta)

# Subtraction
diff_delta = delta1 - delta2
print("Difference in Duration:", diff_delta)
```

**Output:**

```
Total Duration: 2 days, 5:00:00
Difference in Duration: 1 day, 19:00:00
```

### 2. Using timedelta with datetime Objects

You can add or subtract a timedelta from a datetime or date object to get a new datetime or date object.

**Example:**

```
from datetime import datetime, timedelta

now = datetime.now()
print("Current datetime:", now)

# Add 10 days
future_date = now + timedelta(days=10)
print("Date after 10 days:", future_date)

# Subtract 2 hours
past_time = now timedelta(hours=2)
print("Time 2 hours ago:", past_time)
```

**Output:**

```
Current datetime: 2023-10-05 14:30:00
Date after 10 days: 2023-10-15 14:30:00
Time 2 hours ago: 2023-10-05 12:30:00
```

### 3. Calculating Differences Between Dates


Copyright © 2024 HyperionDev. All rights reserved.

Subtracting two datetime or date objects yields a timedelta object representing the time interval between them.

**Example:**

```
date1 = datetime (2023, 10, 5, 14, 30)
date2 = datetime(2023, 11, 5, 16, 45)

difference = date2 - date1
print("Difference:", difference)
print("Days between dates:", difference.days)
print("Total seconds between dates:", difference.total_seconds())
```

**Output:**

```
Difference: 31 days, 2:15:00
Days between dates: 31
Total seconds between dates: 2684700.0
```

**Methods of timedelta**

**1. total_seconds()**

- **Purpose:** Returns the total duration expressed in seconds, including days and microseconds.
- **Syntax:**

```
total_seconds = timedelta_object.total_seconds()
```

- **Example:**

```
delta = timedelta(days=1, hours=2, minutes=30)
print("Total seconds:", delta.total_seconds())
```

**Output:**

```
Total seconds: 93600.0
```

**Practical Applications of timedelta**



Copyright © 2024 HyperionDev. All rights reserved.

### 1. Scheduling Future Events

Calculate future dates and times for events, reminders, or deadlines.

**Example:**

```
from datetime import datetime, timedelta

def schedule_event(days_from_now):
    event_date = datetime.now() + timedelta(days=days_from_now)
    return event_date.strftime("%Y-%m-%d %H:%M:%S")

print("Event scheduled on:", schedule_event(7))
```

### 2. Measuring Elapsed Time

Determine the duration between two events or timestamps.

**Example:**

```
from datetime import datetime

start_time = datetime.now()
# ... some operations ...
end_time = datetime.now()

elapsed_time = end_time - start_time
print("Elapsed time:", elapsed_time)
```

### 3. Validating Date Ranges

Check if a given date falls within a specific time interval.

**Example:**

```
from datetime import datetime, timedelta

def is_within_last_week(date):
    now = datetime.now()
    one_week_ago = now - timedelta(weeks=1)
    return one_week_ago <= date <= now

sample_date = datetime(2023, 10, 2)
print("Is within last week:", is_within_last_week(sample_date))
```



Copyright © 2024 HyperionDev. All rights reserved.

**Key Points to Remember**

- **Immutable Objects:** timedelta objects are immutable; their values cannot be changed after creation.
- **Normalization:** The constructor parameters are normalized into days, seconds, and microseconds internally.
- **Supports Arithmetic:** Can be added to or subtracted from date, datetime, and time objects.
- **Comparison Operators:** timedelta objects support comparison operators (<, <=, >, >=, ==, !=).

- **timedelta Objects:** Represent durations or time intervals.
- **Creation:** Initialize with days, seconds, microseconds, milliseconds, minutes, hours, or weeks.
- **Usage:** Perform date and time arithmetic, calculate durations, schedule future events, and validate time ranges.
- **Integration with datetime:** Works seamlessly with date and datetime objects for powerful date/time manipulation.

By mastering timedelta objects, you can effectively handle time intervals and perform complex date and time calculations in your Python programs.



## Formative Assessment Activity 4

[Formative activity name]

Complete the formative activity in your **Learner Workbook**, as per the instructions from your facilitator.



Copyright © 2024 HyperionDev. All rights reserved.

# Knowledge Topic 5

| Topic Code | KM-03-KT05: |
|---|---|
| Topic | Import, input and output |
| Weight | 10% |

This knowledge topic will cover the following topic elements:

- Concept, definition and functions (KT0501)
- Syntax (KT0502)
- Import (KT0503)
- Input/output (I/O) (KT0504)

After working through this knowledge topic, your competence will be assessed according to the following criterion/criteria:

- IAC0501 Definitions, functions and features of Python import, input and output are understood and explained



Copyright © 2024 HyperionDev. All rights reserved.

## 5.1 Concept, definition and functions (KT0501) (IAC0501)

In Python, the import statement is used to include external modules and libraries into your program. Modules are files containing Python definitions and statements. By importing modules, you can reuse code written by others or organise your code into logical sections.

The import mechanism allows you to access functions, classes, and variables defined in other modules, promoting code reusability and modularity.

### How to Use import

- **Basic Syntax:**

```
import module_name
```

- **Importing Specific Functions or Classes:**

```
from module_name import function_name, ClassName
```

- **Importing with an Alias:**

```
import module_name as alias
```

- **Examples:**

```
import math
print(math.sqrt(16)) # Output: 4.0

from datetime import datetime
now = datetime.now()
print(now)

import numpy as np
array = np.array([1, 2, 3])
```


Copyright © 2024 HyperionDev. All rights reserved.

### Functions of import

- **Code Reusability:** Allows you to use pre-written code, saving time and effort.
- **Modularity:** Helps organise code into modules, making it more manageable.
- **Access to Standard Libraries:** Provides access to Python's extensive standard library.
- **Third-Party Packages:** Enables the use of external packages installed via tools like pip.

### Input in Python

### Concept and Definition

- **User Input:** The input() function in Python allows a program to receive input from the user via the keyboard. It reads a line from the input, converts it to a string (stripping a trailing newline), and returns it.
- **Purpose:** Gathering user input makes programs interactive, enabling them to respond to user data.

### Using input()

- **Basic Syntax:**

```
user_input = input(prompt)
```

- **Parameters:**
    - **prompt (optional):** A string that is displayed to the user before input is taken.
- **Examples:**

```
name = input("Enter your name: ")
```


Copyright © 2024 HyperionDev. All rights reserved.

```
print(f"Hello, {name}!")

age = input("Enter your age: ")
age = int(age) # Convert string input to integer
print (f"You are {age} years old.")
```

### Functions of input()

- **Interactivity:** Makes programs interactive by allowing users to provide data during execution.
- **Dynamic Data:** Enables programs to work with data that is not known until runtime.
- **Customization:** Users can customize the behaviour of programs by providing different inputs.

### Output in Python

### Concept and Definition

- **Displaying Output:** The print() function in Python outputs data to the standard output device (usually the console). It converts the expressions you pass into a string and writes it to the output.
- **Purpose:** Displaying information to the user, debugging, and providing results of computations.

### Using print()

- **Basic Syntax:**

```
print(*objects, sep=' ', end='\n', file=sys.stdout, flush=False)
```

- **Parameters:**
    - ***objects:** One or more objects to be printed.


Copyright © 2024 HyperionDev. All rights reserved.

- **sep (optional):** String inserted between values, default is a space.
- **end (optional):** String appended after the last value, default is a newline.
- **file (optional):** An object with a write(string) method; defaults to sys.stdout.
- **flush (optional):** Whether to forcibly flush the stream.

- **Examples:**

```
print("Hello, World!")

name = "Alice"
age = 30
print("Name:", name, "Age:", age)

# Custom separator and end
print("Python", "Java", "C++", sep=",", ,end=".\n")
```

**Functions of print()**

- **User Communication:** Provides information to the user during program execution.
- **Debugging:** Helps in debugging by displaying variable values and program states.
- **Logging:** Can be used for simple logging purposes.

**Combining Input and Output**

- **Interactive Programs:** By combining input() and print(), you can create interactive programs that take user input, process it, and display results.
- **Example: Simple Calculator**

```
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))
operation = input("Enter operation (+,-, *, /): ")

if operation == "+":
```


Copyright © 2024 HyperionDev. All rights reserved.

```
    result = num1 + num2

elif operation == "-":
    result = num1 - num2

elif operation == "*":
    result = num1 * num2

elif operation == '/':
    result = num1 / num2

else:
   result = "Invalid operation"

print("Result:", result)
```

### Importing Modules for Input and Output

- **Advanced Input/Output:** For more complex input/output operations, such as reading from or writing to files, you can import modules like sys or use built-in functions.
- **Example: Using sys.stdin and sys.stdout**

```
import sys

for line in sys.stdin:
    if line.strip() == 'exit':
        break
    print("You entered:", line.strip(), file=sys.stdout)
```

- **File I/O:**

```
# Writing to a file
with open('output.txt', 'w') as f:
          f.write("Hello, File!\n")

# Reading from a file
with open('output.txt','r') as f:
    content = f.read()
    print(content)
```


Copyright © 2024 HyperionDev. All rights reserved.

- **Import (import):**
    - Allows inclusion of external modules and libraries.
    - Promotes code reusability and modularity.
    - Syntax includes import module, from module import, and aliasing.
- **Input (input()):**
    - Used to get user input from the keyboard.
    - Returns input as a string; can be converted to other types.
    - Essential for interactive programs.
- **Output (print()):**
    - Displays information to the standard output.
    - Accepts multiple arguments, with customizable separators and endings.
    - Useful for communicating with the user and debugging.

## 5.2 Syntax (KT0502) (IAC0501)

The import statement is used to include external modules and libraries into your Python program. This allows you to access additional functions, classes, and variables defined elsewhere.

**Basic Syntax**

1. **Importing an Entire Module**

```
import module_name
```

    - **Usage:** Access module contents with module_name.attribute.

2. **Importing Specific Attributes from a Module**

```
from module_name import attribute_name
```

    - **Usage:** Access the imported attribute directly by its name.

3. **Importing Multiple Attributes**


Copyright © 2024 HyperionDev. All rights reserved.

```
from module_name import attribute1, attribute2
```

4. **Using Aliases**
    - **Module Alias**

```
import module as alias_name
```

    - **Attribute Alias**

```
from module_name import attribute_name as alias_name
```

**Examples**

- **Importing the Entire Module**

```
import math

result = math.sqrt(16)
print(result) # Output: 4.0
```

- **Importing Specific Function**

```
from math import sqrt

result = sqrt(25)
print(result) # Output: 5.0
```

- **Importing Multiple Functions**

```
from math import sin, cos, tan

angle = 0.5
print(sin(angle), cos(angle), tan(angle))
```

- **Using Aliases**


Copyright © 2024 HyperionDev. All rights reserved.

- **Module Alias**

```
import numpy as np
array = np.array([1, 2, 3])
print(array) # Output: [1 2 3]
```

- **Function Alias**

```
from math import factorial as fact
print(fact(5)) # Output: 120
```

### Input in Python

The input() function is used to receive input from the user via the keyboard. It reads a line from input, converts it to a string (stripping a trailing newline), and returns it.

### Syntax

```
variable = input(prompt)
```

- **prompt** *(optional)*: A string displayed to the user before input is taken.

### Examples

- **Basic Input**

```
name = input("Enter your name: ")
print (f"Hello, {name}!")
```

### Output:

```
Enter your name: Alice
Hello, Alice!
```

- **Converting Input to Other Types**


Copyright © 2024 HyperionDev. All rights reserved.

```
age = input("Enter your age: ")
age = int(age) # Converts the input string to an integer
print (f"You are {age} years old.")
```

- **Or directly:**

```
age = int(input("Enter your age: "))
```

- **Calculations with User Input**

```
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))
total num1 + num2

print (f"The sum is: {total}")
```

### Output in Python

The print() function outputs data to the standard output device (screen). It converts the expressions you pass into a string and writes them to the output.

### Syntax

```
print(*objects, sep=' end='\n', file=sys.stdout, flush=False)
```

- ***objects**: One or more expressions to print.
- **sep** *(optional)*: String inserted between values, default is a space ' '.
- **end** *(optional)*: String appended after the last value, default is a newline '\n'.
- **file** *(optional)*: An object with a write(string) method; defaults to sys.stdout.
- **flush** *(optional)*: Whether to forcibly flush the stream.

### Examples

- **Basic Output**

```
print("Hello, World!")
```


Copyright © 2024 HyperionDev. All rights reserved.

- **Printing Multiple Objects**

```
name = "Bob"
age =  25
print("Name:", name, "Age:", age)
```

**Output:**

```
Name: Bob Age: 25
```

- **Using Custom Separator**

```
fruits = ["apple", "banana", "cherry"]
print("Fruits:",'.'.join(fruits))

# Or using sep parameter:
print("Fruits:", *fruits, sep=', ')
```

**Output :**

```
Fruits: apple, banana, cherry
```

- **Using Custom End Character**

```
print("This is the first line.", end=' ')
print("This is the second line.")
```

**Output:**

```
This is the first line. This is the second line.
```

- **Printing to a File**


Copyright © 2024 HyperionDev. All rights reserved.

```
with open('output.txt', 'w') as f:
    print("Writing to a file.", file=f)
```

- **Flushing the Output Buffer**

```
import time

for i in range(3):
    print(i, end='',flush=True)
    time.sleep(1)
```

**Output appears with a 1-second delay between numbers:**

```
0 1 2
```

By understanding the syntax for **Import**, **Input**, and **Output** in Python, you can:

- **Import** external modules to access additional functionality.
- **Receive Input** from users to make your programs interactive.
- **Display Output** to convey information to the user.

These fundamental concepts are essential for building effective Python applications.

## 5.3 Import (KT0503) (IAC0501)

The import statement is used to include external modules and packages into your program. Modules are files containing Python definitions and statements, such as functions, classes, and variables. By importing modules, you can reuse code written by others or organise your own code into separate files for better modularity and maintainability.

**Why Use Import?**

- **Code Reusability:** Allows you to use pre-written code, saving time and effort.
- **Modularity:** Helps in organising code into logical sections.


Copyright © 2024 HyperionDev. All rights reserved.

- **Access to Standard Libraries:** Provides functionalities like math operations, date/time handling, and more.
- **Third-Party Packages:** Enables the use of external libraries installed via package managers like pip.

### Syntax of Importing Modules

#### 1. Importing an Entire Module

```
import module_name
```

- **Usage:** Access module contents using the dot notation: module_name.attribute.

**Example:**

```
import math

result = math.sqrt(16)
print(result) # Output: 4.0
```

#### 2. Importing Specific Attributes from a Module

```
from module_name import attribute_name
```

- **Usage:** Use the imported attribute directly without the module prefix.

**Example:**

```
from math import pi

print(pi) # Output: 3.141592653589793
```

#### 3. Importing Multiple Attributes

```
from module_name import attribute1, attribute2
```

**Example:**


Copyright © 2024 HyperionDev. All rights reserved.

```
from math import sin, cos

angle = 0
print(sin(angle)) # Output: 0.0
print(cos(angle)) # Output: 1.0
```

#### 4. Importing All Attributes (Not Recommended)

```
from module_name import *
```

- **Note:** This imports all public attributes into the current namespace, which can lead to conflicts and reduced code clarity.

**Example:**

```
from math import

print(sqrt(25)) # Output: 5.0
```

#### 5. Using Aliases

- **Module Alias:**

```
import module_name as alias
```

**Example:**

```
import numpy as np

array = np.array([1, 2, 3])
print (array) # Output: [1 2 3]
```

- **Attribute Alias:**

```
from module_name import attribute_name as alias
```


Copyright © 2024 HyperionDev. All rights reserved.

**Example:**

```
from math import factorial as fact

print(fact(5)) # Output: 120
```

### Importing Custom Modules

You can also import modules you've written yourself. Ensure that the module file is in the same directory as your script or in the Python path.

**Example:**

Given a file my_module.py:

```
# my_module.py
def greet (name):
    return f"Hello, {name}!"
```

Importing and using my_module:

```
import my_module

message = my_module.greet("Alice")
print(message) # Output: Hello, Alice!
```

### Packages and __init__.py

- **Packages:** A way of structuring Python's module namespace by using "dotted module names". A package is a directory containing a special file __init__.py, which can be empty or contain initialization code.

**Example Directory Structure (markdown):**

```
my_package/
    __init__.py
    module1.py
    module2.py
```



Copyright © 2024 HyperionDev. All rights reserved.

**Importing from Packages:**

```
from my_package import module1

module1.function_in_module1()
```

**Relative Imports (Within Packages)**

Used when modules within a package need to import other modules from the same package.

- **Syntax:**

```
from. import sibling_module
from .. import parent_module
from .sub_package import module

# Current package
# Parent package
# Subpackage
```

**Example:**

```
# In my_package/module1.py
from. import module2

module2.function_in_module2()
```

**Best Practices**

- **Explicit Imports:** Prefer importing specific attributes to keep the namespace clean.

```
from math import sqrt, pi
```

- **Avoid from module_name import *:** It can cause namespace pollution and make the code less readable.


Copyright © 2024 HyperionDev. All rights reserved.

- **Organise Imports:**
    - **Standard Library Imports:** First group (e.g., import os).
    - **Third-Party Imports:** Second group (e.g., import numpy as np).
    - **Local Application Imports:** Third group (e.g., from my_package import module1).

**Example:**

```
# Standard library imports
import os
import sys

# Third-party imports
import requests

# Local application imports
from my_package import module1
```

- **Use Aliases Judiciously:** Only use aliases when they improve code readability.


Copyright © 2024 HyperionDev. All rights reserved.

**Common Built-in Modules**

- **math:** Mathematical functions like sqrt, sin, cos, pi.
- **datetime:** Classes for manipulating dates and times.
- **os:** Interacting with the operating system.
- **sys:** System-specific parameters and functions.
- **random:** Generate random numbers.
- **re:** Regular expressions.

**Example Using datetime:**

```
from datetime import datetime
current_time = datetime.now()
print("Current Time:", current_time)
```

- **Purpose of Importing:**
  - Reuse existing code.
  - Organise code into modules and packages.
  - Access Python's extensive standard library and third-party libraries.
- **Import Statements:**
  - **import module:** Imports the whole module.
  - **from module import attribute:** Imports specific attributes.
  - **import module as alias:** Imports a module with an alias.
  - **from module import attribute as alias:** Imports an attribute with an alias.
- **Importing Custom Modules:**
  - Place your module in the same directory or in the Python path.
  - Use the import statement as with built-in modules.
- **Packages:**


Copyright © 2024 HyperionDev. All rights reserved.

- Use __init__.py to define packages.
- Organise modules into directories.

- **Best Practices:**
  - Keep imports at the top of the file.
  - Group imports logically.
  - Avoid wildcard imports.

By understanding how to use the import statement effectively, you can leverage existing code libraries, keep your programs organised, and write more efficient and maintainable Python code.

## 5.4 Input/output (I/O) (KT0504) (IAC0501)

In programming, **Input/Output (I/O)** refers to the communication between a computer program and the external world, which could be a user, a file, or another program. In Python, I/O operations allow programs to interact with users or other systems, making them dynamic and interactive.

- **Input**: Data received by the program (e.g., user input from the keyboard).
- **Output**: Data sent from the program (e.g., displaying results on the screen).

**Input in Python**

**Purpose**

- **Interactivity**: Allows a program to receive data from the user during execution.
- **Dynamic Data Handling**: Enables programs to work with data that isn't known until runtime.


Copyright © 2024 HyperionDev. All rights reserved.

### Using the input() Function

The built-in input() function is used to read input from the user.

### Syntax

```
variable = input(prompt)
```

- **prompt** (optional): A string that is displayed to the user before input is taken.

### Example

```
name = input("Enter your name: ")
print (f"Hello, {name}!")
```

### Type Conversion

- The input received is always a string. To use it as another data type, you need to convert it.

### Example

```
age = input("Enter your age: ")
age = int(age) # Convert string input to integer
print (f"You are {age} years old.")
```

### Direct Conversion

```
age = int(input("Enter your age: "))
```

### Common Use Cases

- **Collecting User Data**: Names, ages, preferences.
- **Interactive Programs**: Games, calculators, forms.
- **Data Validation**: Ensuring the user provides valid input.


Copyright © 2024 HyperionDev. All rights reserved.

### Output in Python

#### Purpose

- **User Communication**: Display information, results, or prompts to the user.
- **Debugging**: Print variable values and program states.
- **Logging**: Record program activities.

#### Using the print() Function

The print() function outputs data to the standard output device (usually the screen).

#### Syntax

```
print(*objects, sep='end='\n', file=sys.stdout, flush=False)
```

- ***objects**: One or more expressions to print.
- **sep** (optional): Separator between objects (default is a space).
- **end** (optional): String appended after the last value (default is a newline).
- **file** (optional): Where to send the output (default is sys.stdout).
- **flush** (optional): Whether to forcibly flush the stream.

#### Examples

- **Basic Output**

```
print("Hello, World!")
```

- **Printing Multiple Objects**

```
first_name =  "Alice"
last_name = "Smith"
print("First Name:", first_name, "Last Name:", last_name)
```

- **Custom Separator and End**


Copyright © 2024 HyperionDev. All rights reserved.

```
print("Python", "Java", "C++", sep=", end=".\n")
```

**Output:**

```
Python, Java, C++.
```

- **Printing Without Newline**

```
print("Loading", end="")
print(".", end="")
print(".", end="")
print(".")
```

**Output:**

```
Loading...
```

- **Formatted Output**

```
name = "Bob"
age = 28
print(f"{name} is {age} years old.")
```

### Redirecting Output

- **To a File**

```
with open('output.txt', 'w') as f:
    print("This will be written to a file.", file=f)
```

### Combining Input and Output

By using input() and print() together, you can create interactive programs.

**Example: Simple Calculator**

```
num1 = float(input("Enter first number: "))
```


Copyright © 2024 HyperionDev. All rights reserved.

```
num2 = float(input("Enter second number: "))
operation = input("Enter operation (+, -, *,/")

if operation == '+':
    result = num1 + num2
elif operation == '-':
    result = num1 - num2
elif operation == '*':
    result = num1 * num2
elif operation == '/':
    if num2 != 0:
        result = num1 / num2
    else:
        result = "Cannot divide by zero."
else:
   result = "Invalid operation."

print("Result:", result)
```

### Advanced Input/Output Operations

#### File Input/Output

Python provides built-in functions for reading from and writing to files.

- **Opening a File**

```
# 'r' for reading, 'w' for writing, 'a' for appending
file = open('filename.txt', mode)
```

- **Reading from a File**

```
with open('input.txt', 'r') as file:
       content = file.read()
       print(content)
```

- **Writing to a File**

```
with open('output.txt', 'w') as file:
    file.write("Hello, File!")
```


Copyright © 2024 HyperionDev. All rights reserved.

### File Modes

- **'r'**: Read (default)
- **'w'**: Write (overwrite)
- **'a'**: Append
- **'r+'**: Read and write

### Using sys Module for I/O

The sys module provides access to some variables used or maintained by the interpreter.

- **Reading from Standard Input**

```
import sys

for line in sys.stdin:
    if line.strip() == 'exit':
       break
    print("Input:", line.strip())
```

- **Writing to Standard Error**

```
import sys
print("An error occurred.", file=sys.stderr)
```

### Formatted String Literals (f-strings)

Available from Python 3.6 onwards, f-strings provide a concise and readable way to embed expressions inside string literals.

```
name = "Emily"
score = 95.5
print(f"{name} scored {score} points.")
```

### Best Practices


Copyright © 2024 HyperionDev. All rights reserved.

- **Validate User Input**

Always validate and sanitize user input to prevent errors and security issues.

```
while True:
    try:
        age = int(input("Enter your age: "))
        break
    except ValueError:
        print("Please enter a valid number.")
```

- **Use Context Managers for Files**

The with statement ensures that resources are properly cleaned up after use.

```
with open('file.txt', 'r') as file:
    data = file.read()

# File is automatically closed here
```

- **Handle Exceptions**

Use try-except blocks to handle potential errors.

```
try:
    result = num1 / num2
except ZeroDivisionError:
    print("Cannot divide by zero.")
```

- **Input (input()):**
    - Used to get user input as a string.
    - Syntax: variable = input(prompt)
    - Convert input to other data types as needed.
- **Output (print()):**
    - Used to display information to the user.
    - Syntax: print(*objects, sep=' ', end='\n', file=sys.stdout, flush=False)


Copyright © 2024 HyperionDev. All rights reserved.

- Customize output using sep, end, and string formatting.

- **File I/O:**
  - Read from and write to files using open() with appropriate modes.
  - Use context managers (with statement) for better resource management.

- **Advanced I/O:**
  - Use the sys module for low-level input and output operations.
  - Handle input/output errors gracefully.
- **Best Practices:**
  - Always validate user input.
  - Use exception handling to manage errors.
  - Close files properly or use context managers.



## Formative Assessment Activity 5

Complete the formative activity in your **Learner Workbook**, as per the instructions from your facilitator.



Copyright © 2024 HyperionDev. All rights reserved.



**Share your thoughts**

Please take some time to complete this short feedback **form** to help us ensure we provide you with the best possible learning experience.

# References


Barry, P. (2016). Head first Python (2nd ed.). O'Reilly Media.

Downey, A. B. (2015). Think Python: How to think like a computer scientist (2nd ed.). O'Reilly Media.

Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media.

Matthes, E. (2019). Python crash course (2nd ed.). No Starch Press.

Ramalho, L. (2022). Fluent Python (2nd ed.). O'Reilly Media.

Shaw, Z. A. (2013). Learn Python the hard way (3rd ed.). Addison-Wesley.

Slatkin, B. (2019). Effective Python: 90 specific ways to write better Python (2nd ed.). Addison-Wesley.

Sweigart, A. (2020). Automate the boring stuff with Python (2nd ed.). No Starch Press.





Copyright © 2024 HyperionDev. All rights reserved.